Università degli Studi di Torino

Dipartimento di Informatica



UNIVERSITÀ
DI TORINO

Tesi di Laurea Triennale in Informatica

Un nuovo algoritmo di Machine Learning per la Classificazione di Complessi K: Analisi delle prestazioni e selezione dei parametri liberi.

Relatori:

Prof.ssa. Roberta Sirovich

Dott. Simone Pernice

Candidato:

Marco Giannone

ANNO ACCADEMICO 2023/2024

Abstract

Il presente trattato riguarda lo sviluppo e l'analisi delle prestazioni di un Classificatore, per l'identificazione dei Complessi K (KC), particolari pattern di segnale apprezzabili all' interno di una polisonnografia (elettroencefalogrammi registrati durante il sonno). I dati utilizzati provengono dal Montreal Archive of Sleep Studies (MASS), un vasto archivio che contiene 19 EEG di pazienti durante circa 8 ore di sonno ciascuno.

L'analisi si basa principalmente sulla Topological Data Analysis (TDA), utilizzata per ottenere diagrammi di persistenza attraverso la Lower Star Filtration (LSF) e lo Sliding Window Embedding (SWE) del segnale. Le caratteristiche (Features) utilizzate dal classificatore sono state estratte dai diagrammi di persistenza e da altre proprietà del segnale descritte in letteratura.

Considerando che il numero di KC è significativamente inferiore rispetto ai non KC, è stato necessario creare un dataset bilanciato per addestrare il classificatore. Per questo scopo, abbiamo adottato due strategie: generazione di KC sintetici e l'uso dell' algoritmo SMOTE per la creazione di caratteristiche sintetiche.

Il progetto è stato realizzato in collaborazione con il dottor Giorgio Capaccetta, studente dell'Università degli Studi di Torino, con il quale ho sviluppato una solida collaborazione. Il risultato finale consiste nella creazione di un pacchetto R, che contiene tutto il codice dell'analisi e può essere utilizzato in ambito medico per il riconoscimento dei complessi K attraverso tecniche di machine learning.

Dichiaro di essere responsabile del contenuto dell'elaborato che presento al fine del conseguimento del titolo, di non avere plagiato in tutto o in parte il lavoro prodotto da altri e di aver citato le fonti originali in modo congruente alle normative vigenti in materia di plagio e di diritto d'autore. Sono inoltre consapevole che nel caso la mia dichiarazione risultasse mendace, potrei incorrere nelle sanzioni previste dalla legge e la mia ammissione alla prova finale potrebbe essere negata.

I declare to be responsible for the content I'm presenting in order to obtain the final degree, not to have plagiarized in all or part of the work produced by others and to have cited original sources in a consistent way with current plagiarism regulations and copyright. I am also aware that in case of false declaration, I could incur in legal penalties and my admission to the final exam could be denied.

Contents

1 Introduzione **5**
- 1.1 Polisonnografia e Complessi K . . . 5
- 1.2 R Studio e il linguaggio S4 . . . 6
- 1.3 Cenni di Machine Learning . . . 8
- 1.4 TDA: Analisi Topologica dei Dati . . . 8
- 1.5 Valutare le prestazioni . . . 9

2 Struttura del codice e novità apportate **11**
- 2.1 Flow del codice e organizzazione generale . . . 11
- 2.2 Il problema del bilanciamento del Dataset . . . 13
  - 2.2.1 Creazione di KC-sintetici . . . 14
  - 2.2.2 Bilanciamento con SMOTE . . . 14
  - 2.2.3 Scelta dell'approccio di bilanciamento . . . 15
  - 2.2.4 Addestramento del modello Random Forest . . . 15
- 2.3 Classi e metodi . . . 15
- 2.4 Step4: Persistence Diagram . . . 16
  - 2.4.1 ricerca del minimo . . . 18
  - 2.4.2 SWE e LSF . . . 18
- 2.5 Step5: calcolare le Features . . . 23
- 2.6 Step6: il classificatore . . . 25

3 Un nuovo Classificatore **29**
- 3.1 Cross-Validation . . . 29
- 3.2 K-Fold Cross-Validation . . . 30
- 3.3 Metriche di Valutazione della Random Forest . . . 32
- 3.4 Differenze rispetto all' implementazione classica . . . 34

4 Analisi delle prestazioni e scelta dei parametri liberi **35**
- 4.1 Parametro di Filtraggio . . . 35
  - 4.1.1 Verifica dei risultati tramite *plot* . . . 37
  - 4.1.2 Fattore di rimescolamento . . . 49
- 4.2 Peak_distance . . . 58
- 4.3 Parallelizzazione e Garbage Collection . . . 63

5 Conclusioni **65**

Chapter 1

Introduzione

Questo elaborato vuole riprendere quanto trattato in [1] e in [2] a proposito del riconoscimento automatico dei K-Complexes di cui verranno ripresi i concetti chiave, precisandone alcuni punti e talvolta rielaborandoli. In questo capitolo verranno ripresi i concetti e le definizioni fondamentali per comprendere l'argomento trattato, questo getterà le basi per introdurre i nuovi argomenti frutto del prosieguo del lavoro trattato nei testi citati precedentemente.

1.1 Polisonnografia e Complessi K

Il sonno è una parte fondamentale della vita di un individuo, si stima che ogniuno passi circa un terzo della propria vita dormendo. Analizzando la qualità del riposo notturno si possono ottenere importanti informazioni in merito allo stato di salute di una persona, le informazioni necessarie per condurre una buona analisi si ottengono attraverso un esame chiamato polisonnografia. La polisonnografia, chiamata anche polisonnogramma o analisi del sonno, è un'indagine diagnostica eseguita durante il sonno, per valutare le condizioni di un paziente mentre dorme. Con questo test è possibile effettuare una diagnosi accurata fondamentale per un corretto approccio terapeutico ai diversi disturbi del sonno. [3]. La polisonnografia misura diversi parametri tra i quali l'attività cerebrale, i livelli di ossigeno, il battito cardiaco, la respirazione, il russamento, i movimenti degli arti, talvolta la pressione arteriosa, [5] in particolare l'attività cerebrale è registrata per mezzo di un elettroencefalogramma (EEG) che, attraverso alcuni elettrodi posizionati sul cuoio capelluto, misura l'attività elettrica cerebrale, riproducendola, su uno schermo, sotto forma di una serie di onde.[6]. l' EEG fornisce una serie di segnali sotto forma d'onda che si possono analizzare estraendone i valori che assume in funzione del tempo, se si nota che questi segnali hanno determinate caratteristiche si può parlare di K Complexes (KC), il nome deriva appunto dalla forma che quest'onda asssume. Esplicitiamo quindi queste caratteristiche fornendo la definizione di KC secondo la definizione già descritta in [4]: *Con il termine medico complesso K si intende uno specifico tipo di oscillazione particolarmente ampia e piuttosto rapida del potenziale elettrico neuronale, che si verifica in media una o due volte al minuto durante la fase 2 del sonno non-REM. L'attività elettrica cerebrale viene misurata da appositi macchinari e rappresentata dall'EEG sotto forma di potenziale elettrico in funzione del tempo: i KC corrispon- dono dunque a tratti particolarmente oscillanti di questa funzione. Il concetto di KC non è di facile definizione, soprattutto a causa della grande variabil-*

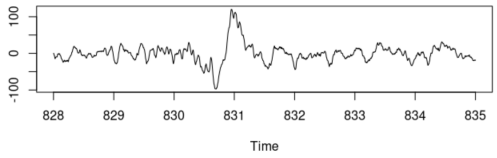


Figure 1.1: forma classica di un KC.

ità che intercorre tra le forme d'onda identificate come tali dagli esperti, spesso non del tutto concor- danti tra loro. Uno dei maggiori svantaggi del riconoscimento ad occhio nudo dei KC – oltre al tempo e all'impegno necessari per esaminare lunghe porzioni di EEG – è proprio la presenza di un'inevitabile componente soggettiva nei processi decisionali, impossibile da eliminare integralmente." Individuare un KC è un operazione difficile a causa della grande variabilità della sua forma, per questo [4] fornisce anche una definizione più tecnica ricavata dalle sue osservazioni, Un complesso K è una forma d'onda costituita da un primo picco negativo netto, immediatamente seguito da un'oscillazione positiva altrettanto (o leggermente meno) ampia ma più lenta. La durata totale di un complesso K è di almeno 0.5s e l'ampiezza relativa da picco a picco supera i 75 μV. Queste caratteristiche sono ben visibili nell' immagine precedente. Studiare i KC è utile perché rappresentano un meccanismo di protezione del cervello, aiutando a mantenere il sonno stabile di fronte a stimoli esterni. Questi complessi sono associati alla sincronizzazione del sonno e svolgono un ruolo cruciale nei processi di memoria e nell'inibizione della veglia. I complessi K possono essere rilevanti nella diagnosi di disturbi del sonno come insonnia, apnea notturna e epilessia notturna. Nelle persone con disturbi neurologici o problemi di regolazione del sonno, un'alterazione della frequenza o della morfologia dei complessi K può indicare anomalie nei processi di inibizione sensoriale durante il sonno. Per approfondimenti legati ai KC si può fare riferimento a [4].

1.2 R Studio e il linguaggio S4

Il contenuto di questo trattato è frutto di un lavoro svolto quasi esclusivamente in **R**, un linguaggio di programmazione e un ambiente di sviluppo utilizzato principalmente per l'analisi statistica, la visualizzazione dei dati e il calcolo scientifico. R deriva dal linguaggio **S**, sviluppato negli anni '70 e '80 presso i laboratori Bell come un linguaggio per l'analisi statistica e la manipolazione dei dati. R è essenzialmente un'evoluzione open-source di S, che ne eredita molte funzionalità e ne amplia le possibilità, diventando uno strumento fondamentale per analisti, statistici e data scientists.

R Studio, invece, è un ambiente di sviluppo integrato (IDE) per R, che semplifica l'uso del linguaggio grazie a un'interfaccia grafica intuitiva e a una serie di funzionalità aggiuntive che facilitano la scrittura, l'esecuzione e il debug del codice. In particolare nel corso dello sviluppo del classificatore l'addestramento è stato fatto con RStudio Server, ovvero un accesso remoto a RStudio su un computer computazionalmente più importante. Con R Studio, gli utenti possono gestire progetti di analisi dati, visualizzare output grafici in tempo reale e integrare pacchetti aggiuntivi per estendere le funzionalità di R. Sebbene R possa essere utilizzato senza un IDE, R Studio rende l'esperienza più fluida e produttiva, specialmente per chi

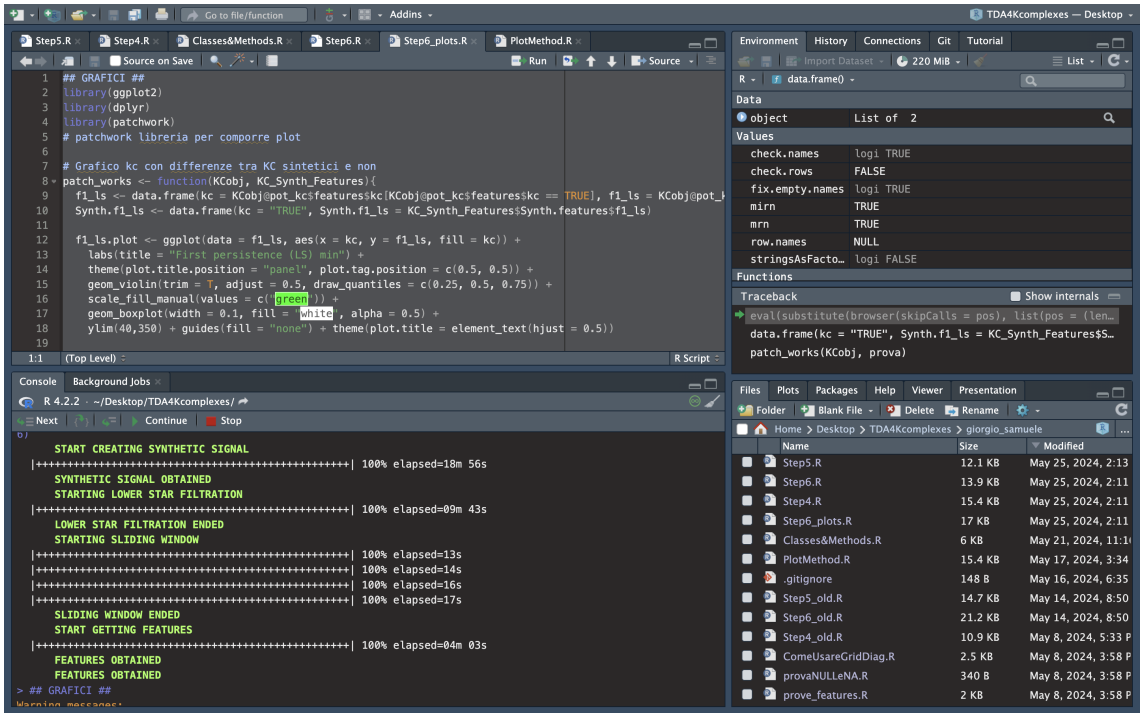


Figure 1.2: una schermata di R studio

lavora con progetti complessi. Un pacchetto in R è un insieme di funzioni, dati e documentazione che estende le capacità del linguaggio stesso. I pacchetti permettono di aggiungere nuove funzionalità a R senza dover riscrivere codice da zero, offrendo strumenti specifici per compiti particolari, come l'analisi statistica avanzata, il machine learning, o la visualizzazione di dati. I pacchetti possono essere installati e caricati dagli utenti con semplici comandi (install.packages() e library()), e ne esistono migliaia disponibili sul repository ufficiale CRAN o su piattaforme come GitHub. In un contesto più generale un pacchetto è una raccolta di software modulare che contiene codice precompilato, librerie, e risorse necessarie per eseguire una determinata funzione o applicazione. I pacchetti facilitano il riutilizzo del codice, la gestione delle dipendenze e la distribuzione di software concetti alla base delle moderne tecniche di sviluppo software. La scelta di sviluppare questo progetto in R e non in altri linguaggi come ad esempio Python, è dovuta al fatto che i pacchetti R sono meno soggetti ad aggiornamenti, evenienza invece molto frequente in Python. Questa scelta permette di risparmiare tempo sulla manutenzione del codice, e in generale permette un codice più stabile e meno soggetto a modifiche esterne, che nell'ambito d'applicazione ovvero quello medico è sinonimo di maggiore affidabilità. Gli oggetti sono fondamentali nel linguaggio R, poiché quasi tutto in R è rappresentato tramite questa struttura dati, inclusi i dati e le operazioni che li manipolano. La struttura di un KC ad esempio è stata replicata tramite un oggetto tramite S4. la distinzione tra i linguaggi S3 e S4, che fanno riferimento a due diversi sistemi di oggetti all'interno di R. S3 è un sistema di oggetti più semplice e flessibile, mentre S4 è più rigoroso e formale, consentendo una maggiore struttura e controllo nelle definizioni degli oggetti e delle loro classi. S4, infatti, permette di definire in modo più dettagliato le proprietà degli oggetti e le relazioni tra di essi, il che è particolarmente utile in ambiti dove è necessaria maggiore precisione, come la programmazione orientata agli oggetti. S4 è un modo per replicare la programmazione

Object-Oriented, tipica di altri linguaggi (Java,C++,etc...) in modo rigoroso e formale,consentendo una maggiore struttura e controllo nelle definizioni degli oggetti e delle loro classi permette di definire in modo dettagliato le proprietà degli oggetti e le relazioni tra di essi.

1.3 Cenni di Machine Learning

Riconoscere i KC è già un compito difficile per l'essere umano, tuttavia seppur difficile anche questo compito è automatizzabile attraverso strumenti informatici che che permettono di insegnare ad una macchina come riconoscerne uno, per far ciò bisogna addestrare il modello, cioè far si che la macchina impari come riconoscere le caratteristiche di un KC. Si è ottenuto questo risultato grazie ad un algoritmo di Machine Learning, ma cos'è il ML? Si riporta la definizione estrapolata dal sito della Oracle: Il Machine Learning (ML) è un sottoinsieme dell'intelligenza artificiale (AI) che si occupa di creare sistemi che apprendono o migliorano le performance in base ai dati che utilizzano. Intelligenza artificiale è un termine generico e si riferisce a sistemi o macchine che imitano l'intelligenza umana. I termini machine learning e AI vengono spesso utilizzati insieme e in modo interscambiabile, ma non hanno lo stesso significato. Un'importante distinzione è che sebbene tutto ciò che riguarda il machine learning rientra nell'intelligenza artificiale, l'intelligenza artificiale non include solo il machine learning. Attualmente, il machine learning è utilizzato ovunque. Quando interagiamo con le banche, acquistiamo online o utilizziamo i social media, vengono utilizzati gli algoritmi di machine learning per rendere la nostra esperienza efficiente, facile e sicura.[7] Per il riconoscimento dei KC la scelta naturale è stata l'implementazione di un classificatore, addestrato a distinguere i KC dai non-KC. Per un risultato ottimale l'algoritmo deve avere a disposizione un dataset. Se il dataset non è bilanciato, ovvero non ha circa lo stesso numero di casi positivi e casi negativi bisogna attuare delle tecniche per bilanciare il dataset per il training.

1.4 TDA: Analisi Topologica dei Dati

Per l'identificazione dei KC sono state applicate tecniche di Analisi Topologica dei Dati (TDA). Un approccio che consente di associare un oggetto geometrico a un insieme di dati, permettendo di studiarne le proprietà strutturali. L'obiettivo è appunto studiarela forma assunta dai dati. Attraverso il pacchetto TDA di R abbiamo ottenuto i diagrammi di persistenza per la Lower Star Filtration (LSF) e la Sliding Window Embedding (SWE) del segnale. I diagrammi di persistenza sono lo strumento principale nella TDA. Aiutano a descrivere e confrontare le forme degli spazi topologici e delle varietà rispetto alle funzioni filtranti. Sono molto utili per analizzare i dati in modo robusto contro il rumore. la SWE invece serve a dare una forma a dei dati sequenziali da dare in pasto a un algoritmo di ML. L'approfondimento teorico della topologia algebrica e del suo impiego nell'analisi dei dati esula dallo scopo di questo lavoro; per una trattazione più dettagliata della teoria e delle basi matematiche, rimandiamo a [4].

1.5 Valutare le prestazioni

Valutare le prestazioni di un classificatore e scegliere correttamente i parametri è fondamentale per garantire un buon funzionamento del modello e previsioni affidabili. Questo processo aiuta a evitare l'overfitting, che si verifica quando il modello impara troppo dai dati di allenamento senza riuscire a generalizzare su dati nuovi, compromettendo così le prestazioni. L'obiettivo principale di un classificatore è proprio la capacità di generalizzare, e testare il modello su un set di test permette di capire quanto sia robusto su dati mai visti prima, considerando soprattutto che il fine ultimo è di essere reso fruibile agli esperti che si occupano di malattie del sonno. Per far ciò ci si è avvalsi di diverse tecniche tra queste la cross-validation ha un importanza centrale all' interno di questo trattato, argomento che verrà approfondito in seguito.

Chapter 2

Struttura del codice e novità apportate

Il codice del Classificatore è piuttosto complesso e lungo, la sua esecuzione segue un flusso e un organizzazione precisa ma articolata, tuttavia nella scrittura del codice si è cercato di preservare la pulizia e la buona leggibilità dello stesso. In questo capitolo verrà brevemente ripreso quanto descritto in [1] e poi ripreso da [2] e verrano illustrate le novità apportate rispetto alla versione descritte nei testi citati.

2.1 Flow del codice e organizzazione generale

Per addestrare un classificatore è importante partire da un grande Dataset, per questo lavoro è stato utilizzato il Montreal Archive of Sleep Studies (MASS) un archivio di dati dedicato alla ricerca sul sonno e ai disturbi legati ad esso. Il MASS È stato creato per offrire alla ricerca una risorsa facilmente accessibile, consentendo la raccolta e la condivisione di grandi quantità di dati provenienti da studi sul sonno. L'archivio contiene dati provenienti da diversi studi polisonnografici raccolti solitamente durante l'arco di un intera notte da qui abbiamo potuto estrarre l'attività cerebrale attraverso i dati dell' EEG. Il MASS è considerato una delle principali risorse per lo studio del sonno, è particolarmente adatto per lo sviluppo di metodi per l'analisi automatizzata e quindi per l'intelligenza Artificiale. Per spiegare quale parte di questo dataset è ha interessato questo lavoro ripoterò quando descritto in [2]:

La porzione di MASS su cui abbiamo basato le nostre analisi è il sottoinsieme 2 del- la coorte 1, ossia MASS-C1/SS2. Questo dataset contiene 19 polisonnografie della durata di circa 8 ore (una notte) eseguite su 19 soggetti giovani e in salute, di cui 8 di sesso maschile (con un'età media di 24.3 anni e compresa tra 19 e 33 anni) e 11 di sesso femminile (età media 23.2 anni, minima 18 e massima 30). Oltre alle poli- sonnografie, sono disponibili per ogni paziente una serie di annotazioni riguardanti la valutazione delle fasi del sonno e della presenza di complessi K e fusi del sonno, tutte operate da un medesimo esperto. Per raggiungere i nostri scopi, il materiale di cui ci siamo effettivamente serviti è il seguente:

- *Di ogni polisonnografia abbiamo conservato solamente il segnale relativo al canale C3 dell'EEG, registrato dall'elettrodo posizionato come da immagine seguente. Tale canale è noto per essere preferibile rispetto agli altri quando*

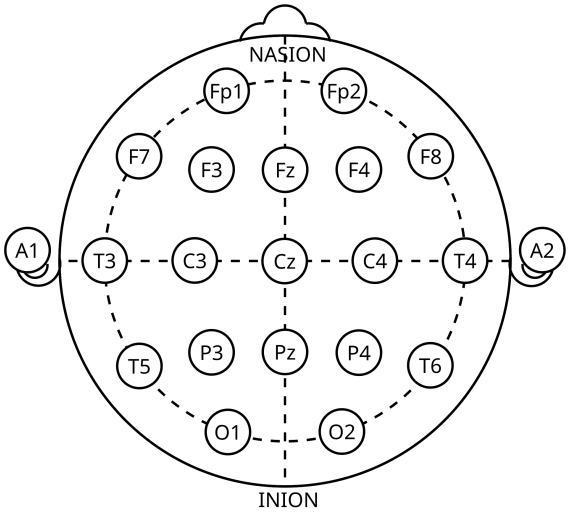


Figure 2.1: Figura 1.3: Schema di un cranio umano osservato dall'alto. Il punto indicato con NASION corrisponde alla radice del naso.

si tratta di osservare i KC e viene usato per sviluppare tutti gli algoritmi di riconoscimento automatico.

- *I file di scoring (classificazione) delle fasi del sonno sono stati necessari per restringere la nostra ricerca dei KC al solo stadio N2. Essi presentano una sequenza di annotazioni, ciascuna riportante il nome di una fase del sonno e riferita ad una delle migliaia di finestre temporali della durata di 20 secondi in cui vengono suddivise la polisonnografie.*

- *Infine, le schede contenenti la classificazione manuale dei complessi K sono state indispensabili per assegnare un'etichetta (KC o non KC) a tutte le forme d'onda selezionate come potenziali complessi K, sia in fase di training che di test. In tali file, per ogni KC rilevato dall'esperto sono riportati un istante di riferimento chiamato onset e la durata del complesso duration.*

Il MASS contiene per ogni paziente tre file che vengono inclusi nel codice grazie all' pacchetto *edfReader* di Comprehensive R Archive Network (CRAN) nel modo seguente :

```
base <- "~/MASS/01-02-0001 Base.edf"
KC <- "~/MASS/01-02-0001 KComplexes_E1.edf"
PSG <- "~/MASS/01-02-0001 PSG.edf"
```

Base contiene le annotazioni sulle fasi del sonno, **PSG** contiene il segnale per intero, **KComplexes_E1** contiene le annotazioni degli esperti dei kc e in particolare contiene il valore *onset* ovvero il secondo in cui è stato rilevato il KC, end il secondo di fine, e lo slot in cui è stato individuato.

La lettura di questi file danno inizio al flusso del codice che è stato così organizzato:

Step 0: In questo step vengono create le classi necessarie per generare i grafici e gestire i dati delle classi. Viene inizializzato l'oggetto principale, `KCobj`, che verrà utilizzato fino alla fine del processo. Questo oggetto contiene tutte le informazioni sul segnale e viene arricchito man mano nei vari step successivi.

Step 1: Qui si trovano i metodi per la visualizzazione grafica dei KC in vari formati.

Step 2: Questo step si concentra sull'identificazione dei minimi nel segnale, utilizzando il metodo `findpeaks` della libreria `pracma`.

Step 3: Qui si associa a ciascun minimo o finestra centrata sui minimi, identificati nello step precedente, l'etichetta `KC/non-KC`, basandosi sulle annotazioni iniziali.

Step 4: In questo step vengono generate la *Lower Star Filtration (LSF)* e la *Sliding Window Embedding (SWE)*.

Step 5: Questo step si occupa dell'estrazione delle features. Il repilogo delle feature estratte e approfondimenti sono trattati in [2].

Step 6: In quest'ultimo step, si affronta il tema del bilanciamento del dataset da utilizzare nel classificatore.

Allo stato attuale il flow del codice prevede diverse operazioni sull' oggetto *Kobj*, risultato finale del nostro codice. In primis si definiscono gli oggetti e le classi, in seguito attraverso lo *step2* si ottengono i minimi da analizzare e si applica la TDA come verrà descritto tra poco. Successivamente si estraggono le features e si può finalmente ricavare la foresta addestrando il classificatore. Tuttavia in questa fase è necessario regolare il nostro dataset.

2.2 Il problema del bilanciamento del Dataset

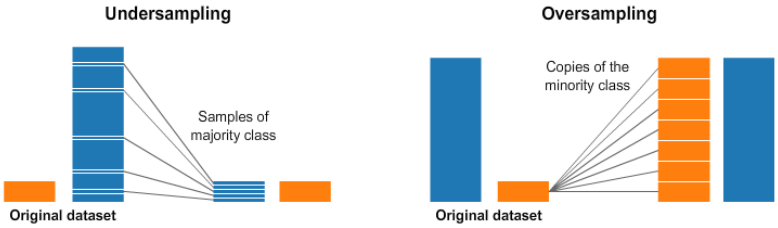


Figure 2.2: Undersampling e Oversampling: il problema di un dataset sbilanciato in favore di una classe o l'altra

Come abbiamo detto, per addestrare un classificatore abbiamo bisogno di un dataset importante in termini di numero delle classi da analizzare. Per far sì che il classificatore riesca a riconoscere correttamente un KC, ha bisogno certamente di casi positivi, ovvero di casi in cui la porzione di segnale analizzata sia effettivamente un KC, ma anche di casi in cui il segnale analizzato non lo è. Nel nostro caso, quest'ultimo è pesantemente sbilanciato, infatti contiene una quantità di KC che non permette una classificazione abbastanza accurata. Per questo motivo, è stato necessario aumentare in maniera *sintetica* il numero di positivi. A tal fine sono

state adottate due tecniche: **la creazione di KC-sintetici** e l'uso dell'algoritmo **SMOTE (Synthetic Minority Over-sampling Technique)**.

2.2.1 Creazione di KC-sintetici

La prima opzione prevede la creazione di segnali sintetici a partire da combinazioni lineari di segnali esistenti. Questo metodo è implementato nella funzione `synthesis`, che genera KC sintetici combinando due segnali esistenti, estratti casualmente dal dataset:

```
synthesis <- function(index){
  n <- sample.int(nrow(Train_KC), size = 2, replace = FALSE)
  c <- 0.50  # Coefficiente di combinazione
  Synth.signal <- lapply(n, synth_frame)
  Synth.KC <- ts((c * as.numeric(Synth.signal[[1]]$frame)
    + (1-c) * as.numeric(Synth.signal[[2]]$frame)),
    frequency = frequency(obj@filtered_signal))
  return(list(Synth.KC=Synth.KC, ...))
}
```

Successivamente, il segnale sintetico viene filtrato(lisciato) per ridurre il rumore, utilizzando il seguente blocco di codice :

```
Synth.KC.Filtered <- stats::filter(Synth.KC, rep(mean,nelem),
  method = "conv", sides = 2, circular = TRUE)
```

Dopo la generazione dei KC sintetici, vengono calcolati i diagrammi di persistenza attraverso la funzione `pers_diag`, che sfrutta la libreria TDA per analizzare la topologia del segnale filtrato e ottenere le caratteristiche necessarie all'addestramento del modello, la vedremo in seguito.

```
res <- get_diag(obj, tau = tau, M = M, shift = shift,
  num_cores = num_cores, testing=testing)
Synth.signal$LS <- res$lower_star
Synth.signal$SW <- res$sliding_window
```

2.2.2 Bilanciamento con SMOTE

Il secondo approccio per il bilanciamento del dataset è l'utilizzo del metodo SMOTE, implementato nella sezione di codice (riga 140) che segue:

```
features <- SMOTE(Train_data, as.factor(Train_data$kc))
```

Questo metodo genera nuovi campioni sintetici attraverso interpolazione tra gli esempi della classe minoritaria, bilanciando il dataset con esempi KC positivi e negativi.

2.2.3 Scelta dell'approccio di bilanciamento

Durante l'esecuzione del codice, l'utente può scegliere tra la generazione di segnali sintetici o l'applicazione del metodo SMOTE. Questa scelta viene gestita con l'input interattivo (riga 36 di `step6`):

```
user.input <- dlgInput("Do you want balance dataset with Synthetic Signal
  or with SMOTE, \n or if you just need features, type features?
  type 1 or 2: ")$res
```

Se l'utente seleziona l'opzione 1, viene eseguita la creazione di segnali sintetici e il calcolo delle caratteristiche. Se l'opzione selezionata è la 2, viene eseguito SMOTE per bilanciare il dataset.

2.2.4 Addestramento del modello Random Forest

Dopo la fase di bilanciamento, i dati bilanciati (sia attraverso segnali sintetici sia tramite SMOTE) vengono utilizzati per addestrare una foresta casuale. Questo modello viene creato utilizzando le caratteristiche estratte dai diagrammi di persistenza o dai segnali sintetici. Il modello finale viene addestrato con la seguente chiamata alla funzione `randomForest`:

```
Synth.data$randomForest <- randomForest(formula.KC ~.,
  data = fix.data_set, importance = TRUE)
```

Il modello così addestrato è in grado di classificare i KC in base alle caratteristiche apprese durante l'allenamento.

2.3 Classi e metodi

Nel codice, i segnali, in accordo coi principi dell' Object-Oriented Programming (OOP) sono tutti rappresentati attraverso la definizione di un oggetto rappresentato da classi, che sfruttano l' ereditarietà di S4.

```
KSInfo <- setClass("KSInfo", slots = list(signal = "ts",
KCs = "data.frame"),
prototype = list(KCs = data.frame(), signal = ts()))
RSInfo <- setClass("RSInfo", slots = list(sleep_stages=
"data.frame"), contains = "KSInfo",
prototype = list(sleep_stages = data.frame()))
MaSInfo <- setClass("MaSInfo",
slots=list(filtered_signal="ts", filter_value="numeric"),
contains = "RSInfo",
prototype = list(filtered_signal = ts(), filter_value =
NA_real_))
```

La creazione dei *KC*-sintetici ha sollevato il problema della rappresentazione come oggetto, difatti i *KC* e i *KC*-Sintetici devono essere indistinguibili per il classificatore, che deve classificarli entrambi come esempi positivi, ma per poter appurare che la classificazione sia corretta è stato importante distinguere il segnale reale da quello sintetico. Rispetto a prima quindi viene adesso introdotta la distinzione attraverso la definizioni di queste due classi:

```
# classe per KC sintetici
SynthInfo <- setClass("SynthInfo", slots = list(synth_data = "list"),
contains = "MaSInfo", prototype = list(synth_data = list(
frame_vec = list(),
LS = list(), SW = list(),features = list())))

SInfo <- setClass("SInfo", slots = list(pot_kc = "list"),
contains = "SynthInfo",
prototype = list(pot_kc = list(min_vec = vector(),
lower_star = list(), sliding_window = list(), features
list())))
```

Questo modifica, a cascata, ha suscitato delle importanti modifiche toccando in particolare il calcolo della LS e della SW. Queste modifiche sono osservabili in *Step4*: La distinzione tra i due casi, ovvero segnale sintetico e segnale non sintetico avviene banalmente attraverso il campo "filtered_signal", infatti il segnale di un kc normale viene filtrato e "lisciato" come spiegato in precedenza, cosa che ovviamente non avviene nei sintetici, che essendo ricavati da un segnale già lisciato non necessita di essere sottoposto a questa procedura.

2.4 Step4: Persistence Diagram

In questo **fondamentale** Step le classi *SInfo* e *SynthInfo*, che rappresentano rispettivamente le informazioni del segnale reale e sintetico sono l'elemento cardine, e come anticipato, l'introduzione di questa distinzione tra i due oggetti ha causato un importante modifica nel codice. Una funzione chiave è *is.KC* che verifica se un minimo corrisponde a un *KC*. Successivamente, la funzione *pers_diag* calcola due tipi di diagrammi di persistenza topologica: la "Lower Star" (che analizza la topologia delle onde attorno a minimi locali) e la "Sliding Window" (che suddivide il segnale in finestre mobili e ne analizza la struttura geometrica). La scelta tra queste due analisi, o entrambe, viene fatta interattivamente dall'utente. Il risultato finale consiste in una serie di diagrammi di persistenza che possono essere usati per identificare le caratteristiche distintive delle onde KC, sia reali che sintetiche, che forniscono supporto alla classificazione automatica del segnale.

Il metodo che segue è una funzione che crea i persistence diagram sui dati sintetici applicando la *Sliding Window Embedding* calcolata precedentemente, è una novità molto importante rispetto alla versione precedente dove il calcolo dei persistence diagram era univoco.

```
setMethod("pers_diag", "SynthInfo", function(obj,
 num_cores = detectCores(),
 M = NA_real_, tau = NA_real_, shift = NULL,
testing = FALSE) { # Sliding Window
```

Si estraggono i frame dalla lista *frame_vec* che si trova in *obj@synth_data$frame_vec*. Ogni frame rappresenta una finestra del segnale sintetico.

```
 frames <- obj@synth\_data\$frame_vec

 # frames<-frames
 min_vec <- list()

 for (i in 1:length(frames)) {
frame <- frames[[i]]

    min_index <- which.min(frame) #indice in cui si verifica il minimo
    min_time <- time(frame)[min_index]
    frame_dim <- tail(time(frame),1) #dimensione finestra

    sx <- min_time - 1
    dx <- frame_dim - min_time

    min_vec[[i]] <- list(frame=frame, sx=sx, dx=dx)
  }

  tmp_synth <- new("SInfo", pot_kc =
  list(min_vec=min_vec, lower_star= NA_real_,
  sliding_window = NA_real_))
  View(tmp_synth)
  # chiamo metodo classe genitrice e salvo risultato in res
  return(pers_diag(obj=tmp_synth, num_cores=num_cores, M=M, tau=tau,
  shift=shift,
  testing=testing))
})
```

Il codice utilizza un ciclo *for* per iterare su tutti i frame contenuti nei dati sintetici. Per ogni frame, il primo passo è individuare l'indice in cui si verifica il valore minimo, usando la funzione *which.min(frame)*. Successivamente, si calcola il tempo esatto in cui questo minimo si presenta all'interno del frame, estraendo l'informazione temporale corrispondente con "*time(frame)[min_ index]*". Una volta identificato il momento del minimo, si procede a determinare la dimensione complessiva del frame, ossia la durata totale della finestra temporale. Questo viene fatto estraendo l'ultimo valore temporale del frame con "tail(time(frame), 1)". A questo punto, si calcolano due variabili: "sx" e "dx", che rappresentano rispettivamente l'espansione temporale a sinistra e a destra rispetto al punto del minimo, solitamente il frame è centrato ovvero il minimo è esattamente al centro della finestra. Questi valori indicano quanto "spazio" temporale si estende prima e dopo il minimo all'interno del frame. Dopo aver iterato su tutti i frame, viene creato un nuovo oggetto di classe "SInfo" da utilizzare sulla classe superiore. Infine, per ciascun frame, viene creata una lista che contiene il frame stesso, insieme ai valori calcolati di "sx" e "dx". Questi dati vengono poi salvati in un nuovo vettore chiamato "min_vec", che conserva tutte le informazioni necessarie per le successive operazioni di analisi topologica. Dopo aver completato l'iterazione su tutti i frame, viene creato un nuovo oggetto di classe "SInfo". Durante la creazione di questo oggetto, viene popolato il campo "pot_kc", che contiene il vettore dei minimi calcolato precedentemente ("min_vec"). Inoltre, vengono inizializzati due campi ("lower_star" e "sliding_window") con il valore

"NA_real_", indicando che non sono ancora stati calcolati i risultati per la filtrazione Lower Star e la Sliding Window Embedding. Questi campi saranno poi riempiti successivamente durante l'analisi. Una volta creato l'oggetto "SInfo", il contenuto può essere visualizzato tramite la funzione "View(tmp_synth)". Questa funzione consente di esplorare e visualizzare in modo interattivo i dati contenuti nell'oggetto appena creato, fornendo all'utente una panoramica delle informazioni elaborate fino a quel punto, inclusi i minimi e le finestre temporali calcolate per ciascun frame. Infine, viene richiamato il metodo "pers_diag" sulla classe genitrice, passando come argomento l'oggetto creato "tmp_synth". Questa chiamata a "pers_diag" è fondamentale, in quanto applica effettivamente le tecniche di analisi topologica, ovvero la Sliding Window Embedding e la Lower Star Filtration, sull'oggetto "SInfo" appena popolato con i minimi. I parametri forniti nella chiamata, come "num_cores", "M", "tau", e "shift", permettono di controllare come l'analisi verrà eseguita; la scelta di questi valori è approfondibile in [4] dove è stata curata la base matematica di questo processo. Alla fine, il metodo restituisce i PS, completando l'intero processo di analisi. Questo risultato conterrà tutte le informazioni relative alla persistenza topologica dei segnali.

2.4.1 ricerca del minimo

```
    is.KC <- function(min = NA_real_, onset =
    NA_complex_, sx = NA_real_, dx = NA_real_){
  if(is.na(min) && is.na(sx) && is.na(dx) && is.NA(onset))
  {
    stop("Please send a valid min, sx & dx parameters")
  }
  kc <- as.logical(sum(min <= (onset + sx - 0.2) & min >=
  (onset - dx + 0.3)))
}
```

La funzione "is.KC" si occupa di determinare se un minimo in un segnale coincide con un K-Complex. Prende in input il minimo rilevato, gli onset dei KC (ovvero il momento di inizio di questi eventi), e i due valori che delimitano l'espansione a sinistra e a destra della finestra temporale attorno al minimo. Se il minimo rientra in questa finestra, la funzione restituisce TRUE, indicando che si tratta di un KC confermato; altrimenti, restituisce FALSE. Ovviamente questa funzione è essenziale per identificare correttamente i KC nei segnali analizzati, ed è il cuore del lavoro del classificatore.

2.4.2 SWE e LSF

```
#VERSIONE OBSOLETA
setMethod("pers_diag", "SInfo", function(obj, sx = 0.8,
dx = 1, # LowerStar
num_cores = detectCores(),
M = NA_real_, tau = NA_real_,shift = NULL){
# SlidingWindow
  user.input <- toupper(dlgInput("Do you want Lower Star
```

```
  of Sliding Window or both? type LS, SW or BOTH: ")$res)
  switch (user.input,
          LS = choose <- 1,
          SW = choose <- 2,
          BOTH = choose <- 3,
          stop("\nINCORRECT INPUT PLEASE TRY AGAIN\n")
  )

#VERSIONE ATTUALE
setMethod("pers_diag", "SInfo", function(obj, sx = 0.8,
dx = 1, # LowerStar
num_cores = detectCores(),
M = NA_real_, tau = NA_real_, shift = NULL,
                                          testing = FALSE)
                                          { # SlidingWindow
  if(!any(is.na(obj@filtered_signal)) && !testing){
    while(TRUE){
      user.input <- toupper(dlgInput("Do you want Lower
      Star of Sliding Window or both? type LS, SW or BOTH: ")$res)
      switch (user.input,
              LS = choose <- 1,
              SW = choose <- 2,
              BOTH = choose <- 3,
              choose <- NA
      )
      if(!is.na(choose))
        break
      else
        cat(crayon::red$bold("\nINCORRECT INPUT PLEASE TRY AGAIN\n"))
    }
  } else
    choose <- 3
```

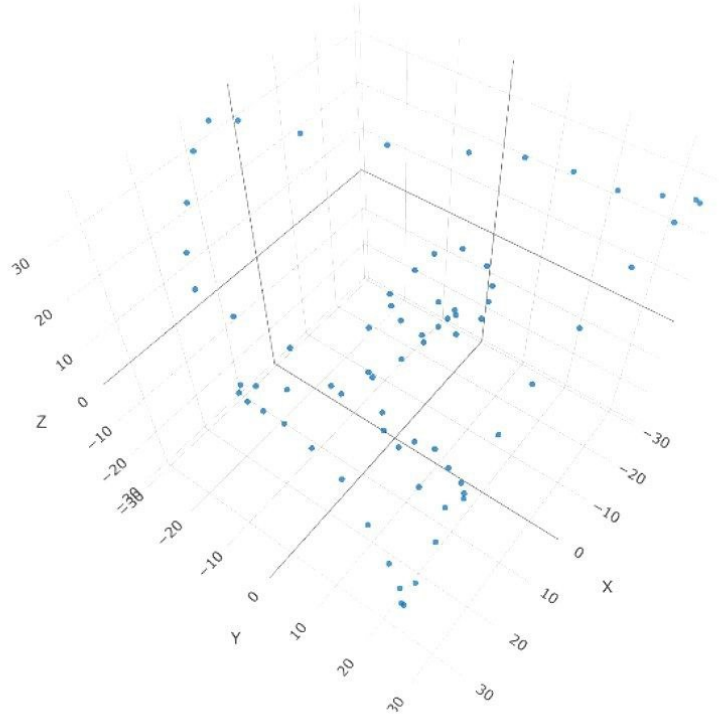


Figure 2.3: Nuvola di punti di un KC

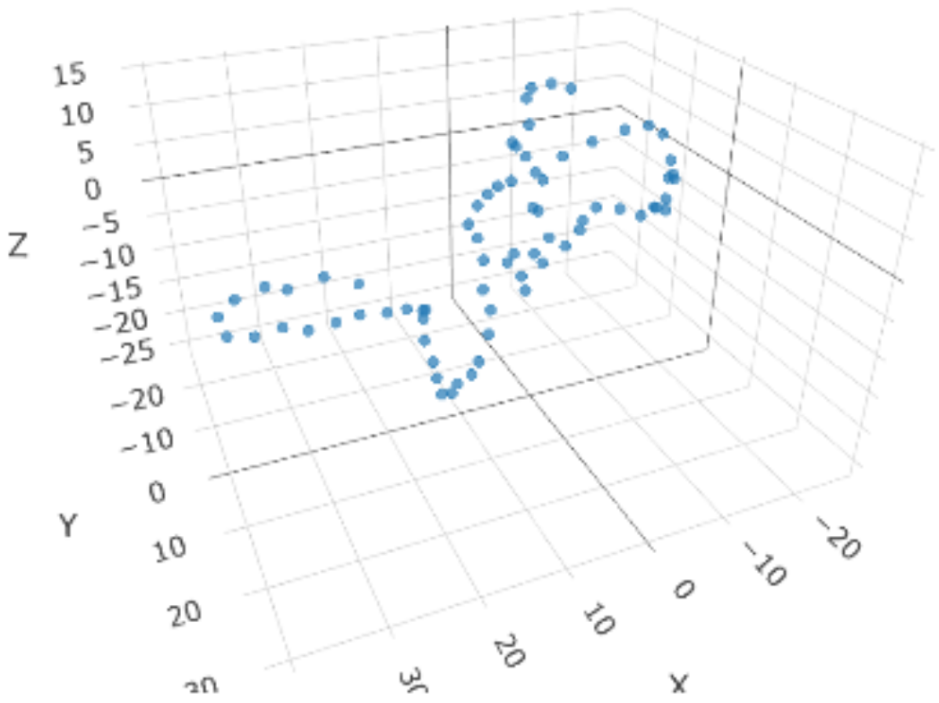


Figure 2.4: Nuvola di punti di un non-Kc

Il metodo *pers_ diag* è un metodo generico definito per la classe SInfo, progettato per applicare la Lower Star Filtration e la Sliding Window Embedding, a un segnale contenuto in un oggetto di questa classe. La logica di esecuzione inizia con l'input dell'utente, che è invitato a scegliere tra tre opzioni: "Lower Star" (LS), "Sliding Window" (SW), o entrambe (BOTH). Se l'utente opta per la Lower Star Filtration, la funzione esegue un'analisi attorno a ciascun minimo, utilizzando una finestra temporale nel segnale. Durante questo processo, viene applicata la filtrazione Lower Star tramite la funzione 'gridDiag()', che genera diagrammi di persistenza per descrivere la topologia del segnale. Per confermare se un minimo corrisponde a un K-Complex. La funzione 'is.KC' è utilizzata in questo contesto. Nel caso in cui l'utente scelga la Sliding Window Embedding, la funzione procede a creare una nuvola di punti (point cloud) attorno ai minimi, rappresentando il segnale in uno spazio multidimensionale. Qui, i parametri 'M' e 'tau' vengono impiegati per definire la dimensione temporale della finestra e la frequenza di campionamento. Successivamente, attraverso la funzione 'ripsDiag()' del pacchetto viene utilizzata per calcolare diagrammi di persistenza, basati sulle distanze tra i punti nel point cloud.

#**LowerStar Filtration versione obsoleta:**

```
if (choose == 1 || choose == 3) {

  lower_star_filtration <- function(min,
  filtered_signal, sx, dx) {
    tmp <- data.frame()

    # Ottengo il diagramma di persistenza tramite
    gridDiag
    diagram <- gridDiag(FUNvalues = as.numeric(-
    window(filtered_signal, start = min - sx, end = min
    + dx)),
                        library = 'GUDHI')

    kc <- is.KC(min, obj@KCs$onset, sx, dx) #
    as.logical(sum(min <= (obj@KCs$onset + sx - 0.2)  &
    min >= (obj@KCs$onset - dx + 0.3)))
    tmp <- list(sx = sx, dx = dx, diagrams =
    diagram$diagram, kc = kc)

    return(tmp)
  }
  cat(crayon::green$bold("      STARTING LOWER STAR
  FILTRATION\n"))
  obj@pot_kc[["lower_star"]] <-
  pblapply(obj@pot_kc[["min_vec"]],
  lower_star_filtration, obj@filtered_signal, sx, dx,
  cl = clust)
  cat(crayon::green$bold("      LOWER STAR FILTRATION
  ENDED\n"))
```

```
  if(choose == 1){
    stopCluster(clust)
    return(obj@pot_kc)

  }
}
```

LowerStar Filtration versione attuale:

```
if (choose == 1 || choose == 3) {

  #controllo se il segnale è stato filtrato, quindi se
  si tratta di un sintetico
  lower_star_filtration <- function(min_vec,
  filtered_signal) {
    # se segnale originale devo calcolare finestra
    frame, altrimenti ho già la finestra
    if(!any(is.na(obj@filtered_signal))){
      frame <- -window(filtered_signal, start =
      min_vec$min - min_vec$sx, end = min_vec$min +
      min_vec$dx)
      # diciamo che c'è un KC nella finestra che viene
      lavorata dalla LS se c'è un onset ragionevolmente
      nella finestra
      kc <- is.KC(min_vec$min, obj@KCs$onset,
      min_vec$sx, min_vec$dx)
    }else{
      frame <- -min_vec$frame
      kc <- TRUE
    }
    tmp <- data.frame()
    # Ottengo il diagramma di persistenza tramite
    gridDiag
    diagram <- gridDiag(FUNvalues = as.numeric(frame),
    library = 'GUDHI', maxdimension=0)

    tmp <- list(sx = min_vec$sx, dx = min_vec$dx,
    diagrams = diagram$diagram, kc = kc)
    gc()
    return(tmp)
  }


  cat(crayon::green$bold("      STARTING LOWER STAR
  FILTRATION\n"))

  if(!any(is.na(obj@filtered_signal))){
```

```
    min_vec <- lapply(obj@pot_kc$min_vec, function(min)
    {list(min=min, sx=sx, dx=dx)})
    pot_kc <- list(min_vec = obj@pot_kc$min_vec,
    lower_star = list(), sliding_window = list())
  } else{
    min_vec <- obj@pot_kc$min_vec
    pot_kc <- list(lower_star = list(), sliding_window
    = list())
  }

  pot_kc[["lower_star"]] <- pblapply(min_vec,
  lower_star_filtration, obj@filtered_signal , cl =
  clust)

  cat(crayon::green$bold("      LOWER STAR FILTRATION
  ENDED\n"))

  if(choose == 1){ # se l'utente vuole solo LS è
  inutile che io faccia il controllo dopo
    stopCluster(clust)

    return(pot_kc)
  }
```

Nella versione precedente, il codice assumeva che il segnale fosse sempre disponibile e quindi filtrato in tempo reale. La funzione *lower_star_filtration* lavorava direttamente sui dati grezzi, senza verificare lo stato del segnale. Al contrario, nella versione attuale, viene introdotto un controllo per verificare se il segnale è già stato filtrato ed è quindi è un sintetico. Se il segnale esiste, si procede normalmente, calcolando una finestra intorno ai minimi rilevati nel segnale. Se invece il segnale è sintetico o per qualche motivo non disponibile, la funzione utilizza una finestra predefinita già presente nei dati, migliorando la flessibilità e la velocità di calcolo. Se il segnale è sintetico, viene considerato automaticamente che il KC sia presente, mentre nel caso di segnali reali viene eseguito un controllo più rigoroso utilizzando la funzione *is.KC*. Questo approccio semplifica la gestione dei segnali generati artificialmente. I risultati dell'elaborazione sono gestiti in modo più organizzato nella versione attuale. Se nella versione obsoleta i risultati venivano inseriti direttamente in una struttura predefinita, nella nuova versione il processo è più modulare. Ora i dati sono separati in modo chiaro tra *lower_star* e *sliding_window*, migliorando la comprensibilità del codice e facilitando eventuali future estensioni.

2.5 Step5: calcolare le Features

```
    \begin{verbatim}
setMethod("extract_features", "SInfo", function(obj,
num_cores = detectCores(),
learning = TRUE) {
```

```
# check su numero di core disponibili prova
num_cores <- min(num_cores, (detectCores()-1))

# imposto i cluster in base al numero di core disponibili nel sistema
clust <- makeCluster(num_cores)

clusterExport(clust, varlist = c("obj"), envir = environment())
```

Nello Step5 ci si occupa di sviluppare le *features* per il classificatore, le features servono a catturare le proprietà rilevanti del segnale in per poterle utilizzare nell'algoritmo del classificatore, e per altre eventuali analisi statistiche. Sono cruciali perché un modello di machine learning non può operare direttamente sui dati grezzi in modo efficiente. Estrarre le features giuste può migliorare notevolmente l'accuratezza e la capacità del modello di distinguere tra diversi tipi di eventi, ad esempio, K-complexes e non K-complexes. Il metodo chiave del codice è *extract_features*, che estrae delle feature dai dati sintetici o reali contenuti nell' oggetto *SInfo*. La funzione può operare in modalità di "*learning*" (addestramento) o semplicemente estrarre feature senza alcuna supervisione. Se l'oggetto di input è del tipo SynthInfo (Ksintetiche), vengono manipolati dati KC sintetici. Se l'oggetto è del tipo SInfo (informazioni reali o sintetiche già filtrate), la funzione estrae effettivamente le feature.

```
    if(!any(is(obj@pot_kc$min_vec[[1]]) == "ts"))
  featuresDataFrame <- data.frame(min =
  obj@pot_kc$min_vec)
else
  featuresDataFrame <- data.frame()

persistence_features <- function(index) {
  # per utilizzare Step5 coi sintetici faccio in modo
  di controllare cosa passo così da discriminare e
  comportarmi in modo differente
  if(!any(is(obj@pot_kc$min_vec[[index]]) == "ts")) #
  se == TS ==> è synth_kc ==> ramo else, ho già TS,
  altrimenti devo ritagliarmi il frame di mio interesse
    frameTime <- window(obj@filtered_signal,
                        start =
                        obj@pot_kc$min_vec[[index]] -
                  obj@pot_kc$lower_star[[index]]$sx,
                  end =
                  obj@pot_kc$min_vec[[index]] +
                  obj@pot_kc$lower_star[[index]]$dx)
  else
    frameTime <- obj@pot_kc$min_vec[[index]]
```

La discriminazione tra segnale sintetico e non è ovviamente frutto delle modifiche alla classe, infatti nella versione precedente il codice calcolava ogni volta la finestra d'interesse. Cosa che adesso non viene fatta perchè la dimensione del frame è stabilita a monte.

```
  #VERSIONE OBSOLETA:

featuresDataFrame <- data.frame(min =
obj@pot_kc$min_vec)

persistence_features <- function(index) {
  cat(crayon::green$bold("      START GETTING
  FEATURES\n"))
  cat(crayon::green$bold(paste("      index = ", index,
  sep = "")))
  frameTime <- window(obj@filtered_signal,
                      start =
                      obj@pot_kc$min_vec[[index]] -
                      obj@pot_kc$lower_star[[index]]$sx,
                      end = obj@pot_kc$min_vec[[index]]
                      +
                      obj@pot_kc$lower_star[[index]]$dx)
```

La funzione successiva è *persistence_features* che è responsabile dell'estrazione delle *features* da ogni segmento del segnale, le caratteristiche del segnale catturate riguardano ad esempio la persistenza più duratura del picco, la durata in termini temporali, e la pendenza del **minimo** o del **massimo**, tutte informazioni rilevanti che costiuiscono le caratteristiche del **KC**. Per approfondimenti sul calcolo delle features fare riferimento a [2].

2.6 Step6: il classificatore

Ritorniamo all' inizio del capitolo in cui si parlava del bilanciamento del dataset. Lo *Step 6* rappresenta una fase cruciale del processo, in cui vengono applicati algoritmi di machine learning, nello specifico la *Random Forest*, per la classificazione. In questo step, si sfruttano i metodi descritti precedentemente per bilanciare il dataset, e succesivamente sfruttando i passi dello *Step_ 5* per estrarre le caratteristiche (*features*) dai segnali, si addestra il modello di classificazione.

Una *Random Forest* è un ensemble di alberi decisionali, dove il modello finale è ottenuto combinando le previsioni di molteplici alberi. Questa tecnica offre maggiore robustezza rispetto al singolo albero decisionale, migliorando la capacità predittiva e riducendo il rischio di overfitting. L'overfitting è un comportamento indesiderato che si verifica quando il modello fornisce previsioni molto accurate per i dati di addestramento ma non sufficienti per riconoscere i nuovi dati, in pratica il classificatore è impreciso.

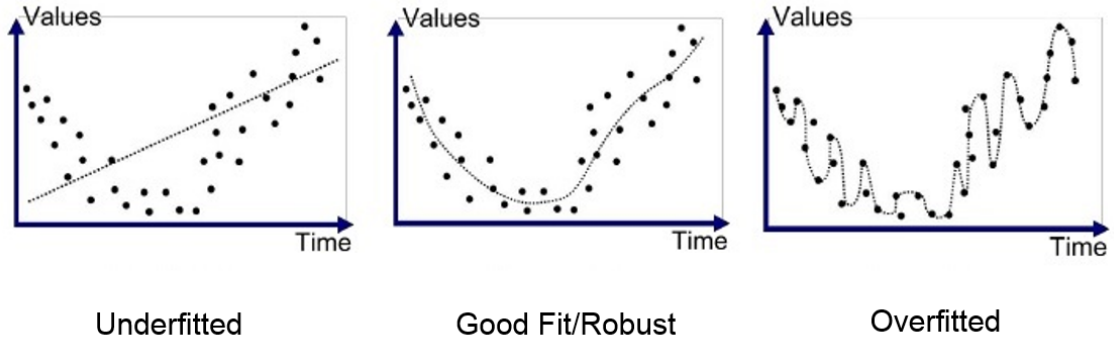


Figure 2.5: differenze sul fitting dei dati

Di seguito, il codice che implementa il metodo *make_forest* per creare la foresta:

```
setGeneric("make_forest", function(obj, ...)
standardGeneric("make_forest"), signature = "obj")
setMethod("make_forest", "SInfo",
function(obj, Synth.signal = NULL, Train_data = NULL,
Train_KC = NULL, features =
NULL, balance = TRUE, M= NA_real_, tau = NA_real_,
shift = NULL, num_cores =
detectCores(), testing = FALSE)
```

In questo Step scegliamo se bilanciare il dataset scegliendo tra SMOTE e creazione dei Sintetici. successivamente calcoliamo i Diagrammi di Persistenza e ricaviamo le features.

```
    cat(crayon::green$bold("      START CREATING
    SYNTHETIC SIGNAL\n"))
        Synth.signal <- list(frame_vec = list(),
        synth.signal = list(), index_signal = list(), LS
        = list(), SW = list())
        frame_list <- pblapply(1:Synth.number_KC,
        synthesis, cl = clust)

        # popolo la mia lista di segnali sintetici
        Synth.signal$frame_vec <- lapply(frame_list,
        function(x){x$filtered})
        Synth.signal$synth.signal <- lapply(frame_list,
        function(x){x$Synth.KC})
        Synth.signal$index_signal <- lapply(frame_list,
        function(x){x$index_signal})

        cat(crayon::green$bold("      SYNTHETIC SIGNAL
        OBTAINED\n"))
```

L'addestramento della foresta avviene con il pacchetto `randomForest` di R, che consente di ottenere il modello finale. Questo modello è costruito considerando l'importanza delle variabili, che consente di identificare quali features contribuiscono maggiormente alla classificazione.

Al termine di questo step, il classificatore è pronto per essere utilizzato sui dati di test o per la validazione su dati non visti in precedenza.

```
  #### RANDOM FOREST ####
  set.seed(1)
  cat(crayon::green$bold("      START CREATING THE
  FOREST\n"))
  # creo formula
  formula.KC <- factor(ifelse(features$kc == TRUE,
  "POSITIVE", "NEGATIVE"))

  # aggiusto dataset
  fix.data_set <- subset(features, select = -c(min,
  kc))

  # OTTENGO LA FORESTA
  Synth.data$randomForest <- randomForest(formula.KC
  ~., data = fix.data_set, importance = TRUE)
  cat(crayon::green$bold("      FOREST CREATED\n"))
}
```

Chapter 3

Un nuovo Classificatore

Il principale paradigma trattato in questo manoscritto riguarda il testing di un nuovo algoritmo, che è in particolare un nuovo Classificatore, in questo capitolo verrà illustrato il funzionamento di questo nuovo algoritmo, e i risultati ottenuti in termini di performance.

3.1 Cross-Validation

Questo codice è progettato per automatizzare il processo di addestramento e valutazione del modello di Random Forest utilizzando il pacchetto `caret`, uno degli strumenti più versatili e popolari in R per il machine learning. La prima parte del codice riguarda l'ottenimento dei dati, la loro preparazione per il training del modello e la creazione della Random Forest con validazione incrociata:

```
data <- list.forest[[1]]$Synth.features
data$kc <- factor(data$kc)
```

Il pacchetto `caret` semplifica la costruzione di questo modello e include funzioni avanzate per la quale richiede la creazione di un oggetto di controllo di train con una funzione personalizzata per il calcolo delle metriche:

```
mySummaryFunction <- function(data, lev = NULL, model =
NULL){
  mmetric(y = as.factor(data$obs), x = data$pred, metric
  = c("ACC", "PRECISION", "TPR"))
}
```

Questa funzione è utilizzata per calcolare specifiche metriche di performance che vedremo dopo. Queste metriche sono comunemente utilizzate per valutare la qualità di un classificatore, fornendo una panoramica complessiva della sua efficacia.

Successivamente si imposta il TrainControl che è il metodo della libreria necessario per indirizzare la classificazione. ctrl <- trainControl(method = "cv", summaryFunction = mySummaryFunction)

Il parametro `method = "cv"` indica che il metodo di validazione scelto è la *cross-validation*. La Cross-Validation(CV) [9] è *" è un approccio essenzialmente semplice e ragionevole per stimare l'accuratezza predittiva dei modelli di previsione. La CV è sviluppata in molte fonti standard sulla modellazione di regressione e "apprendimento automatico".* In parole più semplici è una tecnica utilizzata per valutare le

prestazioni di un modello di machine learning in modo affidabile. L'idea è quella di dividere il dataset in più parti per fare in modo che ogni esempio venga utilizzato sia per l'addestramento che per il test del modello La tecnica più comune è il K-Fold Cross-Validation, che vedremo tra poco. L'addestramento del modello Random Forest si effettua utilizzando il metodo `RRF` (Random Regularized Forest), che è una variante della Random Forest:

```
    rfFit <- train(
  kc ~ .,
  data = data,
  method = "RRF",
  trControl = ctrl
)
```

Questa parte del codice definisce una funzione che automatizza l'intero processo descritto sopra, rendendo facile eseguire il fitting del modello su più dataset:

```
fittingCaret <- function(list.data){
data <- list.data$Synth.features
data$kc <- factor(data$kc)
mySummaryFunction <- function(data, lev = NULL, model
= NULL){
mmetric(y = as.factor(data$obs), x = data$pred,
metric = c("ACC", "PRECISION", "TPR"))
  }

ctrl <- trainControl(method = "cv", number=20,
summaryFunction = mySummaryFunction)

rfFit <- train(
kc ~ .,
data = data,
method = "rf",
trControl = ctrl
  )
}
```

3.2 K-Fold Cross-Validation

Il nostri dataset vengono quindi suddivisi attraverso la tecnica della **cross-validation**. I **fold** rappresentano le partizioni del dataset utilizzate per addestrare e testare un modello di machine learning in modo iterativo e sistematico. Uno degli approcci più utilizzati è la K-fold cross-validation, in cui l'intero dataset viene suddiviso in K sottoinsiemi (fold) di dimensioni approssimativamente uguali.

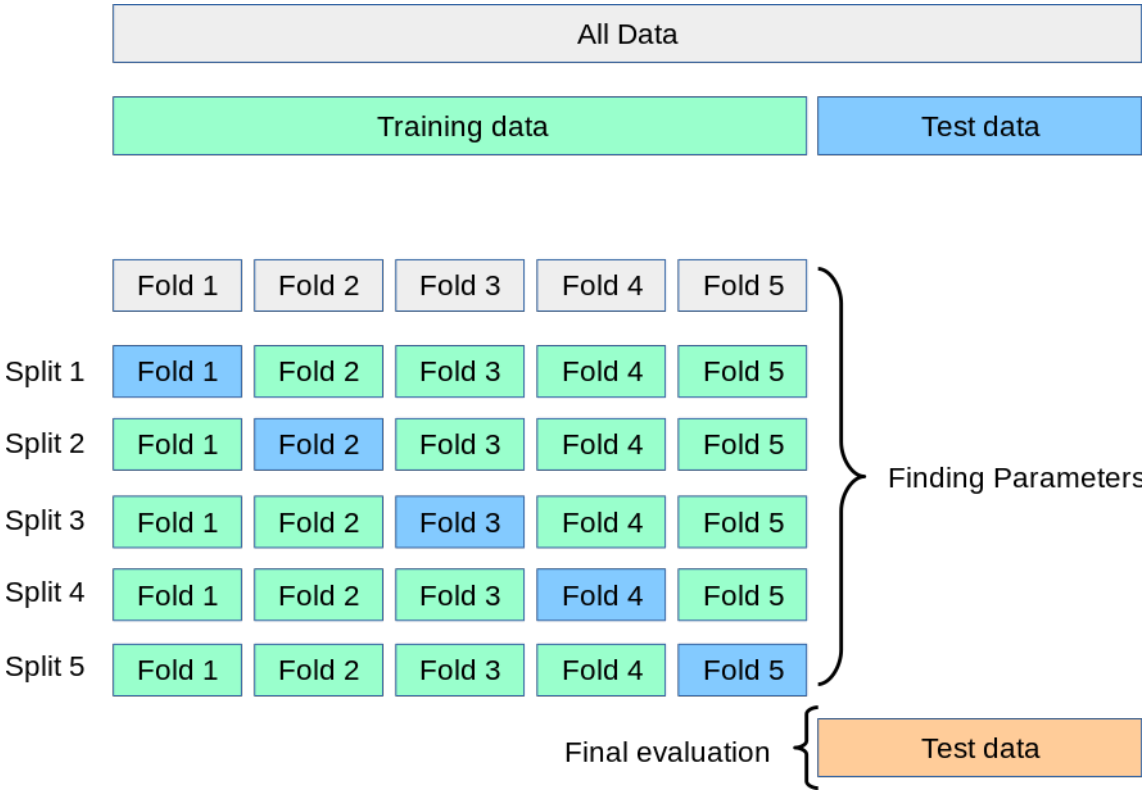


Figure 3.1: il modello effettua un training utilizzando k-1 fold [8]

Durante ogni iterazione del processo di K-fold cross-validation, uno dei K fold viene assegnato al ruolo di set di **test** per valutare le prestazioni del modello, mentre i restanti K-1 fold vengono utilizzati come set di **training** per l'addestramento del modello stesso. Questo ciclo di addestramento e validazione si ripete esattamente K volte, alternando il fold utilizzato per il test in ogni iterazione, fino a che ciascun fold non è stato utilizzato una volta come test set e K-1 volte come training set. Al termine delle K iterazioni, le prestazioni del modello vengono aggregate su tutti i fold. Le metriche ottenute (come accuratezza, precisione, ecc.) vengono quindi mediate per fornire una stima complessiva delle prestazioni del modello. Questo approccio permette di ottenere una valutazione robusta e affidabile, poiché tutti i campioni del dataset vengono utilizzati sia per l'addestramento sia per il test, seppure in momenti differenti. Se prendiamo ad esempio la 5-fold cross-validation, in cui il dataset viene diviso in cinque parti. In ogni iterazione, uno dei cinque fold viene usato come set di test, mentre i restanti quattro fold vengono usati per il training. Questo approccio garantisce che ciascun esempio del dataset venga testato esattamente una volta, e allo stesso tempo il modello viene valutato su cinque diversi sottogruppi del dataset.

La scelta del numero di fold, K, è un parametro importante che bilancia accuratezza ed efficienza computazionale. Un numero elevato di fold, come nella 10-fold cross-validation, fornisce una stima più accurata delle prestazioni generali del modello, ma aumenta, ovviamente, il costo computazionale. Al contrario, un numero basso di fold riduce il tempo di calcolo a scapito di una valutazione potenzialmente meno precisa. In casi estremi ogni singolo campione nel dataset viene utilizzato come set di test in K iterazioni, con il resto dei dati usato per l'addestramento. L'uso della cross-validation, e in particolare dei fold, è quindi essenziale per una

valutazione affidabile della capacità di generalizzazione di un modello su dati non visti, riducendo al contempo il rischio di **overfitting**.

3.3 Metriche di Valutazione della Random Forest

Una rappresentazione utile delle prestazioni di un modello di classificazione è fornita dalla **matrice di confusione**. Questa matrice fornisce una tabella che mostra il numero di predizioni corrette e errate suddivise per ciascuna classe. La matrice di confusione ha la seguente struttura:

	Predetto Positivo	Predetto Negativo
Reale Positivo	TP	FN
Reale Negativo	FP	TN

Dalla matrice di confusione, è possibile calcolare direttamente le metriche di valutazione precedentemente descritte. Ad esempio: - Il numero di veri positivi (TP), veri negativi (TN), falsi positivi (FP) e falsi negativi (FN) può essere estratto direttamente, consentendo un calcolo semplice di accuracy, precision, e recall. La matrice di confusione è uno strumento essenziale per comprendere come un modello sta performando, poiché non solo mostra il numero totale di errori, ma fornisce anche dettagli su quali classi sono state confuse tra loro. Tuttavia per valutare le prestazioni di un classificatore, è fondamentale utilizzare diverse metriche che forniscono una comprensione completa della sua efficacia. Le metriche accennate prima e più comunemente utilizzate includono:

- **Mtry**: determina quante feauture considera il modello a ogni passo durante la costruzione degli alberi decisionali. Un valore più basso rende l'addestramento più casuale, ogni albero considera meno informazioni, mentre un valore più alto consente a ciascun albero di prendere in considerazione più caratteristiche.
- **Accuracy (ACC)**:
  - **Definizione**: La percentuale di predizioni corrette rispetto al numero totale di predizioni.
  - **Formula**:
$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$
  - Un'accuratezza alta indica un modello che predice correttamente la maggior parte dei risultati.
- **Precision**:
  - **Definizione**: La proporzione di risultati positivi predetti che sono realmente positivi.
  - **Formula**:
$$\text{Precision} = \frac{TP}{TP + FP}$$
  - Una precisione alta significa che il modello ha un basso tasso di falsi positivi, il che è cruciale in scenari in cui i falsi positivi sono costosi o dannosi.

- **Recall (o True Positive Rate, TPR)**:
  - **Definizione**: La proporzione di positivi reali che sono stati correttamente identificati dal modello.
  - **Formula**:
$$\text{Recall (TPR)} = \frac{TP}{TP + FN}$$
  - Un alto valore di recall indica che il modello è efficace nell'identificare i casi positivi, riducendo al minimo i falsi negativi.

- **Matthews Correlation Coefficient (MCC)**:
  - **Definizione**: Una misura della qualità di una classificazione binaria che considera tutte le categorie.
  - **Formula**:
$$\text{MCC} = \frac{TP \times TN - FP \times FN}{\sqrt{(TP + FP)(TP + FN)(TN + FP)(TN + FN)}}$$
  - L'MCC varia da -1 a +1, dove +1 indica una classificazione perfetta, 0 indica una classificazione casuale e -1 indica una classificazione completamente errata.

L'obiettivo della cross-validation, ottenute le metriche è scegliere proprio l'*mtry* ottimale in base a una metrica specifica, nel caso questa non fosse specificata, *Caret* sceglie in autonomia basandosi sulla *Accuracy* migliore.

```
> print(cv20)
Random Forest

27634 samples
   14 predictor
    2 classes: 'FALSE', 'TRUE'

No pre-processing
Resampling: Cross-Validated (20 fold)
Summary of sample sizes: 26252, 26252, 26252, 26253, 26252, 26252, ...
Resampling results across tuning parameters:

  mtry  ACC       PRECISION1  PRECISION2  TPR1      TPR2
   2    94.33306  96.17715    92.64471    92.34276  96.32344
   8    94.36199  95.93689    92.90435    92.65397  96.07011
  14    94.27151  95.89340    92.77222    92.50920  96.03389

ACC was used to select the optimal model using the largest value.
The final value used for the model was mtry = 8.
```

Figure 3.2: Risultato della CV con Caret, oltre alle metriche, in output viene indicato l' mtry ottimo scelto in base alla metrica.

3.4 Differenze rispetto all' implementazione classica

Nel confronto tra i due approcci emergono differenze significative sia in termini di performance che di flessibilità. L'approccio basato sulla cross-validation (CV) tramite *caret* garantisce una stima più robusta delle prestazioni del modello, grazie alla sua capacità di fornire una valutazione accurata anche in presenza di dati variabili. Dividendo il dataset in più sottogruppi e testando il modello su ciascuno di essi si riduce il rischio di overfitting. Questo metodo però richiede un notevole impegno computazionale, poiché il modello deve essere addestrato nuovamente per ogni fold. Di conseguenza, pur risultando in modelli generalmente più stabili e affidabili, il tempo di esecuzione può aumentare sensibilmente, soprattutto con dataset di grandi dimensioni.

Dall'altra parte, l'approccio che utilizza segnali sintetici per generare foreste casuali affronta il problema dello squilibrio nei dati, migliorando la rappresentatività del dataset. Questo può aumentare l'efficacia del modello nel riconoscere pattern complessi senza richiedere molteplici cicli di cross-validation. L'uso di dati sintetici offre inoltre una maggiore capacità di generalizzazione su nuovi dati, tuttavia richiede attenzione nelle scelte metodologiche, poiché eventuali imprecisioni nella generazione dei segnali sintetici possono compromettere le prestazioni del modello. In termini di efficienza, questo approccio può risultare più rapido rispetto alla CV, ma la qualità dei risultati dipende fortemente dalla validità dei dati sintetici e dalle ipotesi su cui si basa il processo. La CV offre maggiore stabilità al costo di tempi di esecuzione più lunghi, mentre l'uso di segnali sintetici rappresenta una soluzione più flessibile e veloce, ma è certamente un approccio meno standardizzato e più complesso. La scelta tra le due implementazioni non è necessariamente esclusiva, ma necessita di approfondimenti ulteriori che non sono oggetto di questo trattato.

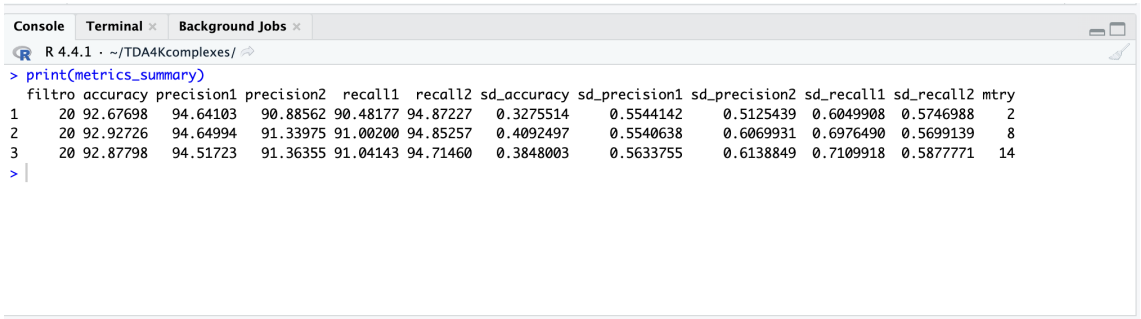


Figure 3.3: Metriche ottenute con una cv a 20 fold

Chapter 4

Analisi delle prestazioni e scelta dei parametri liberi

Nella costruzione di un classificatore, è cruciale testare e ottimizzare i parametri e le variabili libere coinvolte nel processo. Ogni fase del pipeline di analisi, dalla pre-elaborazione del segnale all'estrazione delle caratteristiche, fino alla classificazione, può essere influenzata in modo significativo dalla scelta di determinati parametri. Tra questi, il filtraggio del segnale è uno degli aspetti fondamentali, poiché incide direttamente sulla qualità del dato grezzo su cui si basano tutte le successive analisi. Per approfondimenti sulla logica matematica trattata in questo capitolo fare riferimento a [4].

4.1 Parametro di Filtraggio

Nel contesto di questo progetto, il filtro agisce lisciando il segnale, riducendo il rumore presente nei dati grezzi. Tuttavia, non esiste un valore universale di filtro che funzioni in ogni circostanza, poiché il bilanciamento tra preservare le caratteristiche distintive del segnale e ridurre il rumore di fondo è estremamente delicato. Se il filtro è troppo "aggressivo", cioè se viene impostato un valore troppo alto, si rischia di perdere informazioni essenziali che caratterizzano le oscillazioni EEG, compromettendo la capacità del classificatore di riconoscere i K-Complexes o altre caratteristiche rilevanti. D'altro canto, un filtro troppo debole potrebbe lasciare troppo rumore nel segnale, confondendo il modello di classificazione e diminuendo le prestazioni complessive.

La sperimentazione su diversi valori di filtro permette quindi di trovare il giusto compromesso tra questi due estremi. Testando valori di filtraggio differenti, si può verificare quale configurazione ottimizza il riconoscimento delle caratteristiche desiderate (ad esempio, i minimi locali nel segnale) mantenendo il rumore a livelli accettabili. Questo approccio permette di migliorare la qualità dei diagrammi di persistenza e, di conseguenza, delle feature estratte dal segnale, che rappresentano la base su cui il classificatore costruisce il proprio modello. L'adattamento del filtro ai dati specifici in esame è quindi un passaggio critico per garantire che il classificatore sia in grado di generalizzare efficacemente, mantenendo alte prestazioni anche su dati mai visti prima.

```
#codice di testing del segnale
filter_vec <- c(20, 30, 50)
```

I parametri di test utilizzati sono nell' ordine *20,30,50* idealmente il parametro *30* dovrebbe rappresentare il punto di ottimo ma i confronti dei plot possono dare un idea della differenza che c'è tra un parametro e un altro e fornire delle indicazioni in merito quindi dimostrare che una lisciatura più aggressiva, quindi un parametro con valore più alto possa essere ottimale o meno.

```
func <- function(filter){
  # ottengo i minimi
  cat("ottengo minimi", "\n")
  #step2
  KCobj@pot_kc$min_vec <- getMin(KCobj, 0.5, 20)

  # ottengo diagrammi di persistenza
  cat("ottengo persistenza", "\n")
  #step4
  KCobj@pot_kc <- pers_diag(KCobj, M = 2, tau =
  48/256, shift = 5/256, testing = TRUE)

  # ottengo features
  cat("ottengo features", "\n")
  #step5
  KCobj@pot_kc$features <- extract_features(KCobj)

  # ottengo dataset su cui allenare e testare
  foresta
  set.seed(2)
  features <- KCobj@pot_kc$features    # prendo le
  features
random_data <- sample(1:nrow(features),nrow(features)*0.7)    #
ottengo la suddivisione dei data tra training e
test (70-30)
  Train_data <- features[random_data,]    # ottengo
  i dati per il training set  (70%)
  Test_data <- features[-random_data,]    # ottengo
  i dati per il test set  (30 %)
  # aumento il numero di KC nel Train_data per
  avvicinarmi ad un 50-50 di KC e non-KC
  Train_KC <- Train_data[Train_data$kc == TRUE,]  #
  ottengo tutti i kc dal train set

  # ottengo foresta
  cat("ottengo foresta", "\n")
  #step6
  forest <- make_forest(KCobj, Train_data =
  Train_data, Train_KC = Train_KC, M = 2, tau =
  48/256, shift = 5/256, testing=TRUE)

  return(list(KCobjs=KCobj, Forests=forest))
```

```
}

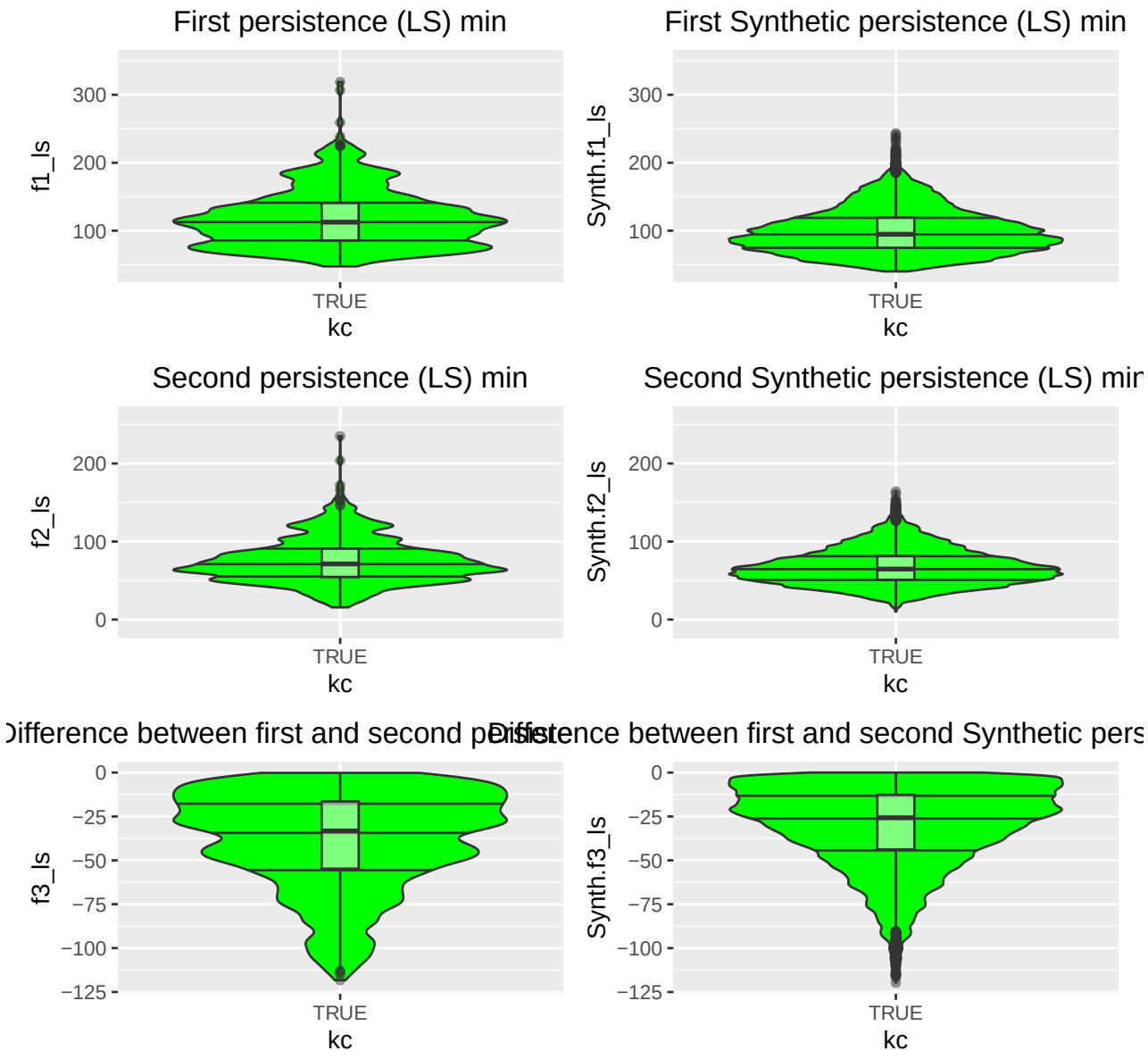
list.synth <- lapply(filter_vec, func)
```

Il codice allegato ci da anche un idea di come il flusso del codice sia strutturato, eseguendo nell' ordine le varie funzioni che compongono il training del classificatore. Si può verificare nel dettaglio le bontà dei risultati ottenuti dai vari parametri attraverso dei Violin plot.

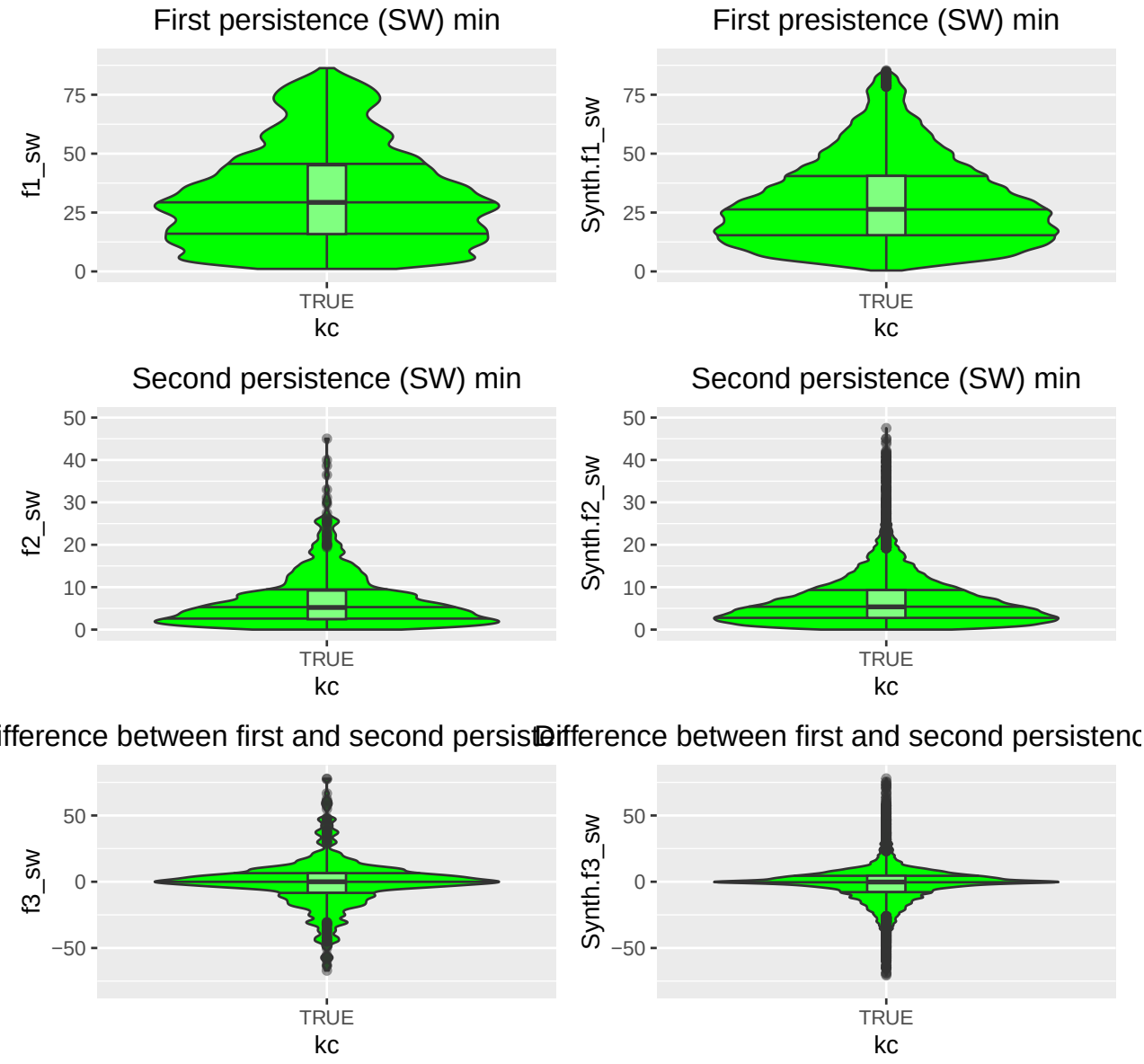
4.1.1 Verifica dei risultati tramite *plot*

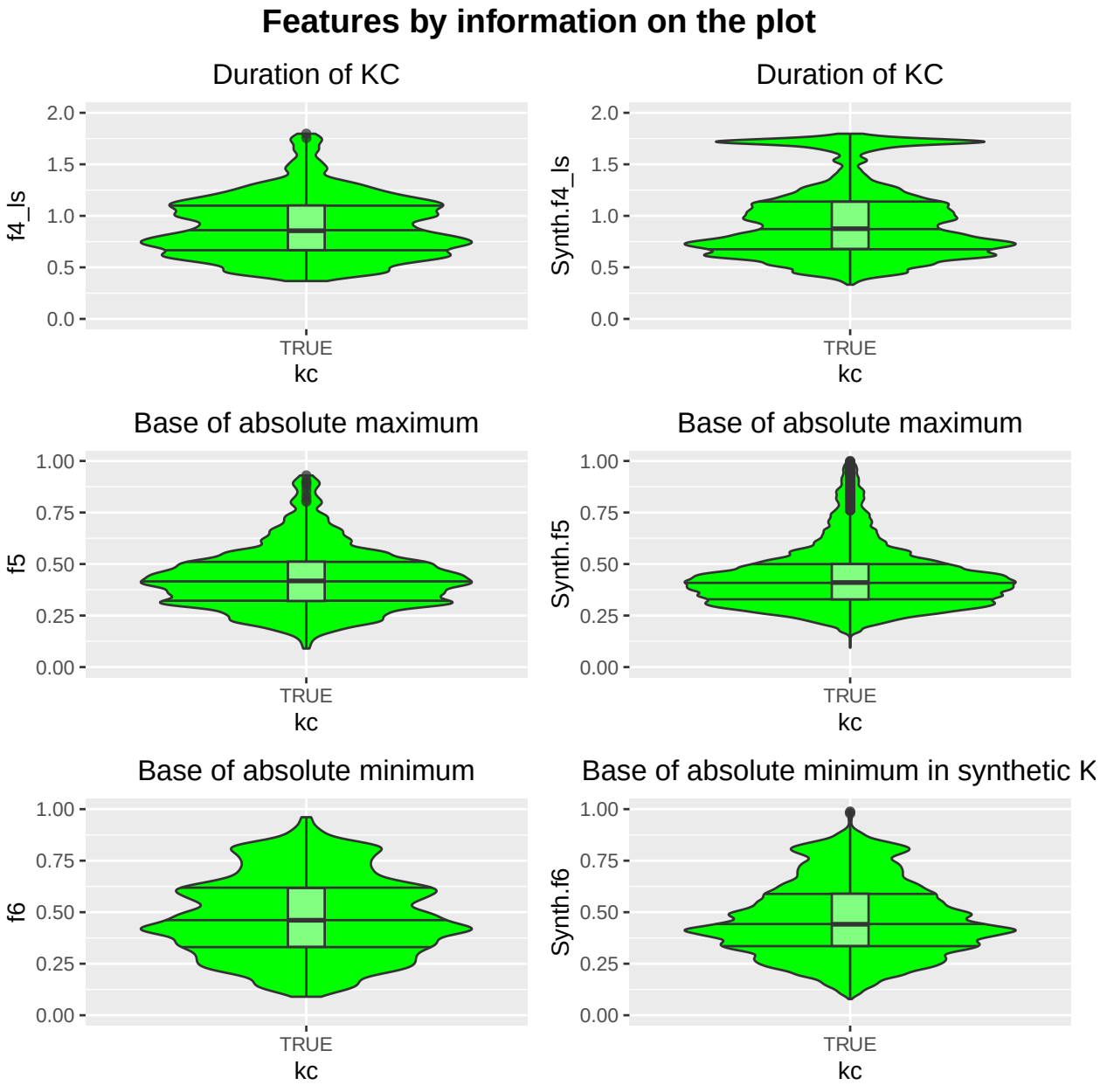
Il processo di testing del parametro di filtraggio è essenziale per garantire che il segnale venga adeguatamente preparato per l'analisi. In un progetto di classificazione così articolato, l'ottimizzazione dei parametri, come il filtraggio, gioca un ruolo chiave nel migliorare la precisione e la robustezza del modello finale. Si possono verificare i risultati dei test attraverso dei plot. Nelle prossime 4 pagine vengono confrontate le Features, oltre che **LS** e le **SW** dei KC originali e di quelli sintetici con filtro impostato a *20*. Come si evince, i grafici ottenuti sono, correttamente, molto simili. Nelle successive 4 pagine il confronto avviene con filtro impostato a 30. Infine si può invece osservare il confronto delle metriche ottenute in Cross-Validation estraendo le metriche attraverso il pacchetto *rminer*.

Features by persistence diagrams

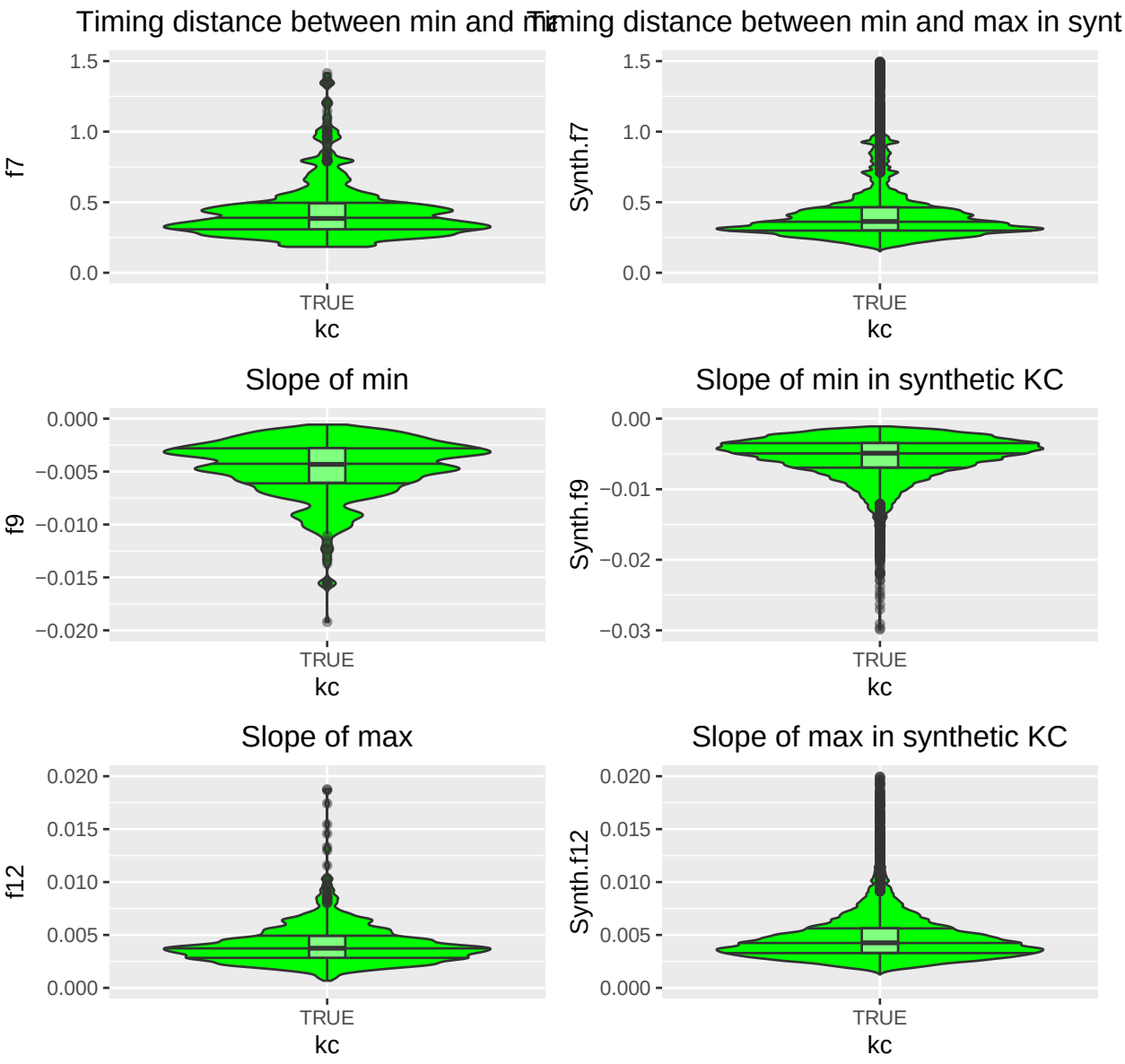


Features by persistence diagrams

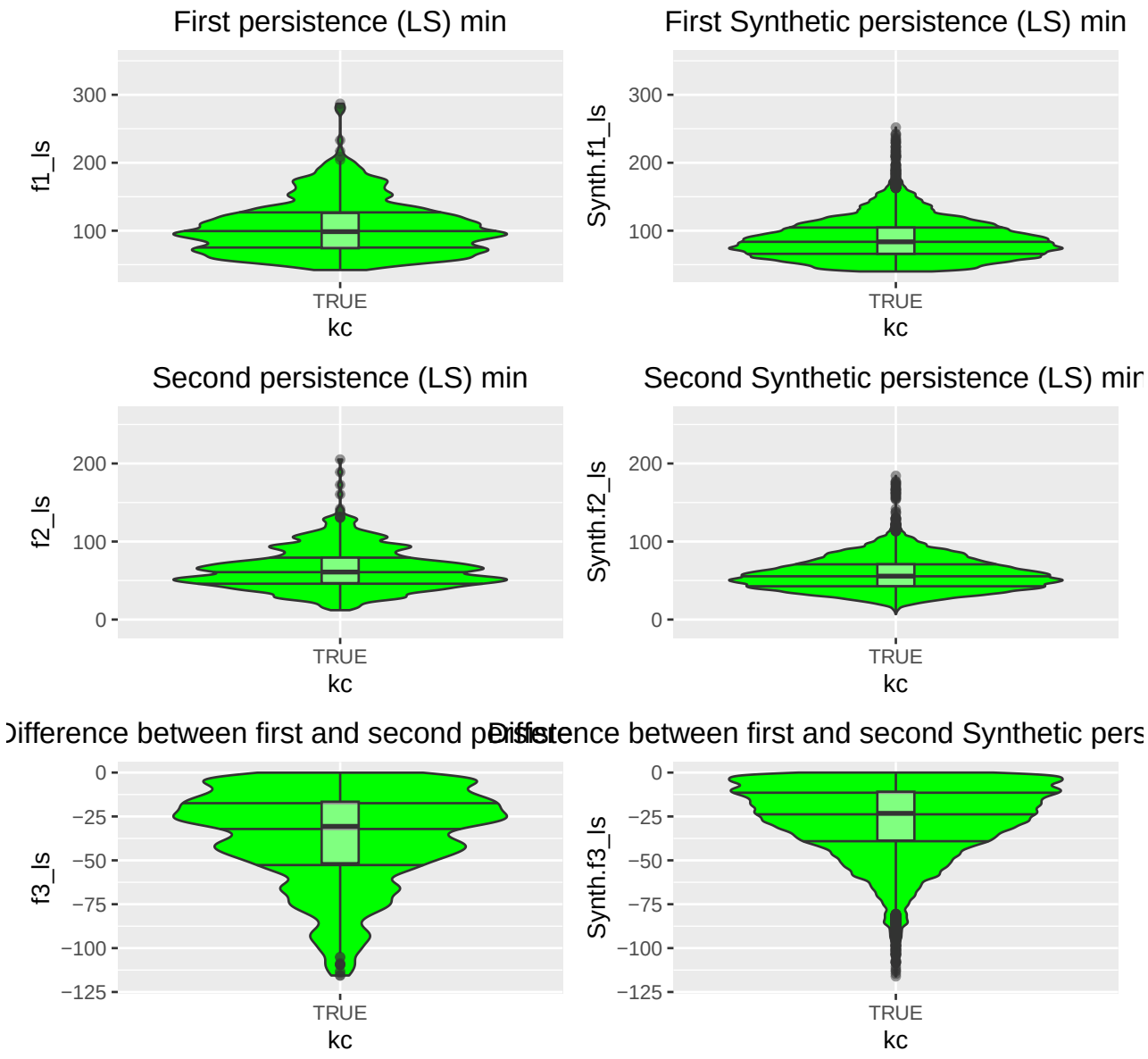



Features by information on the plot
Duration of KC
f4_ls
2.0
1.5
1.0
0.5
0.0
TRUE
kc
Duration of KC
Synth.f4_ls
2.0
1.5
1.0
0.5
0.0
TRUE
kc
Base of absolute maximum
f5
1.00
0.75
0.50
0.25
0.00
TRUE
kc
Base of absolute maximum
Synth.f5
1.00
0.75
0.50
0.25
0.00
TRUE
kc
Base of absolute minimum
f6
1.00
0.75
0.50
0.25
0.00
TRUE
kc
Base of absolute minimum in synthetic K
Synth.f6
1.00
0.75
0.50
0.25
0.00
TRUE
kc

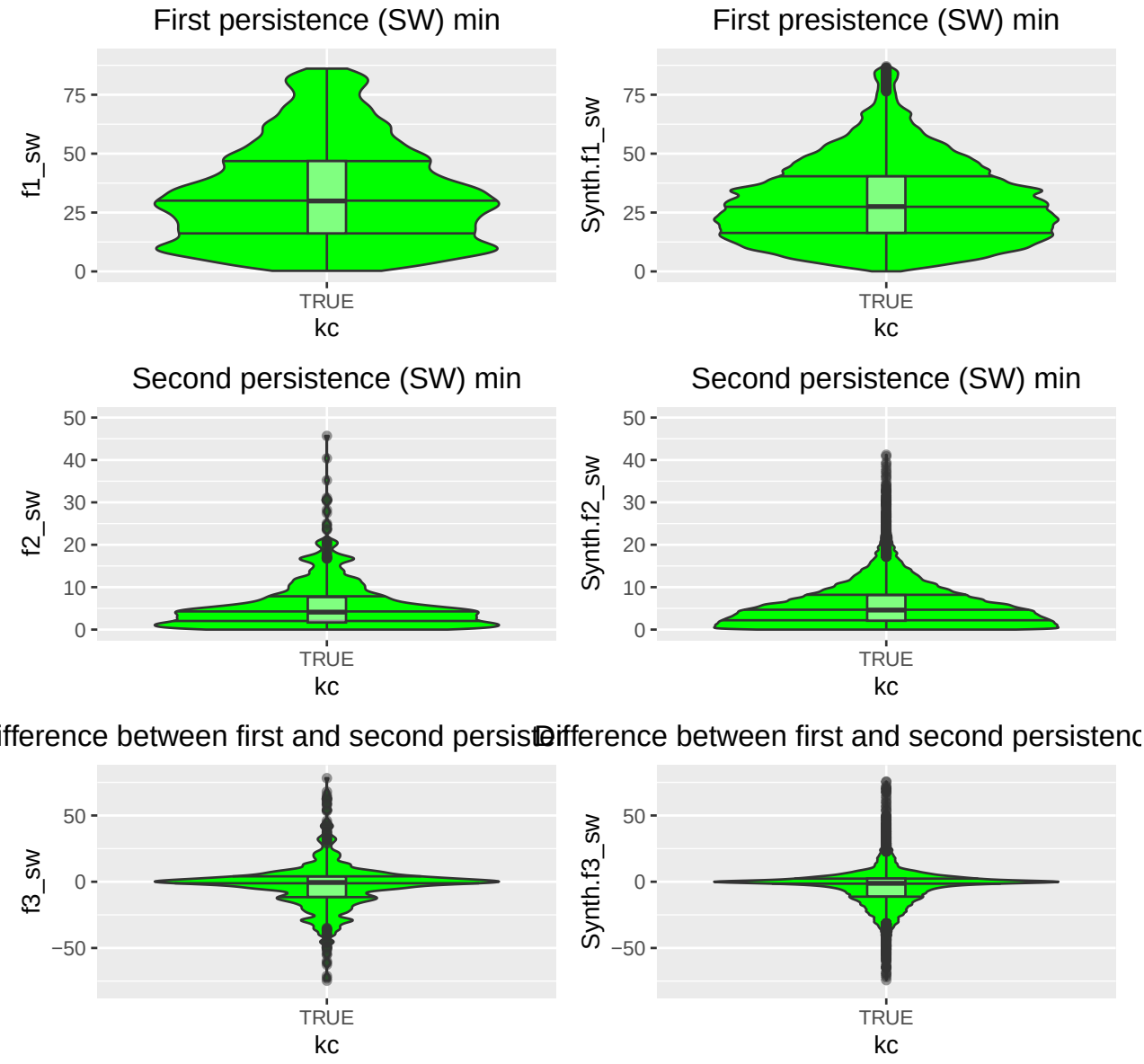
Features by information on the plot



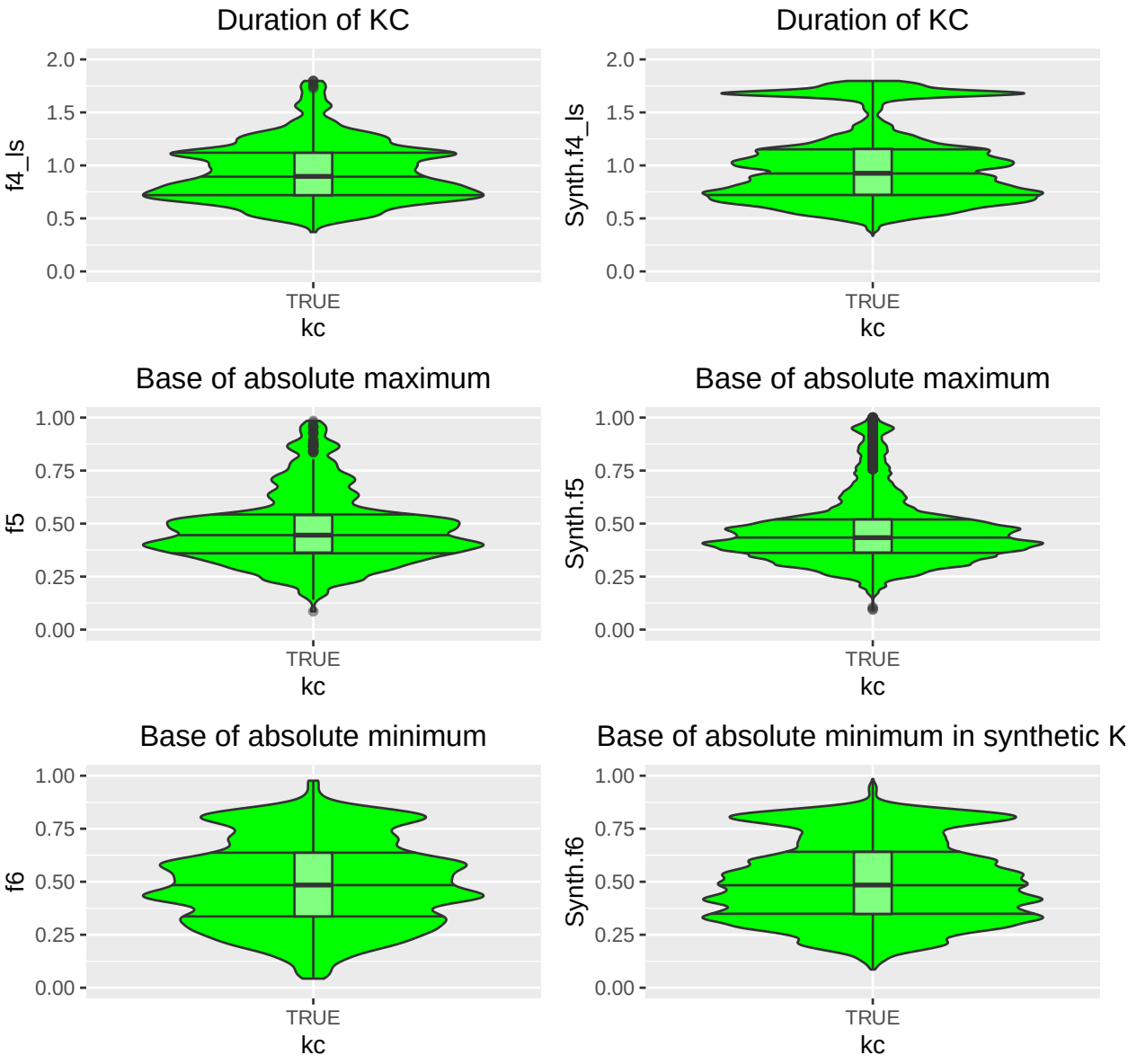
Features by persistence diagrams



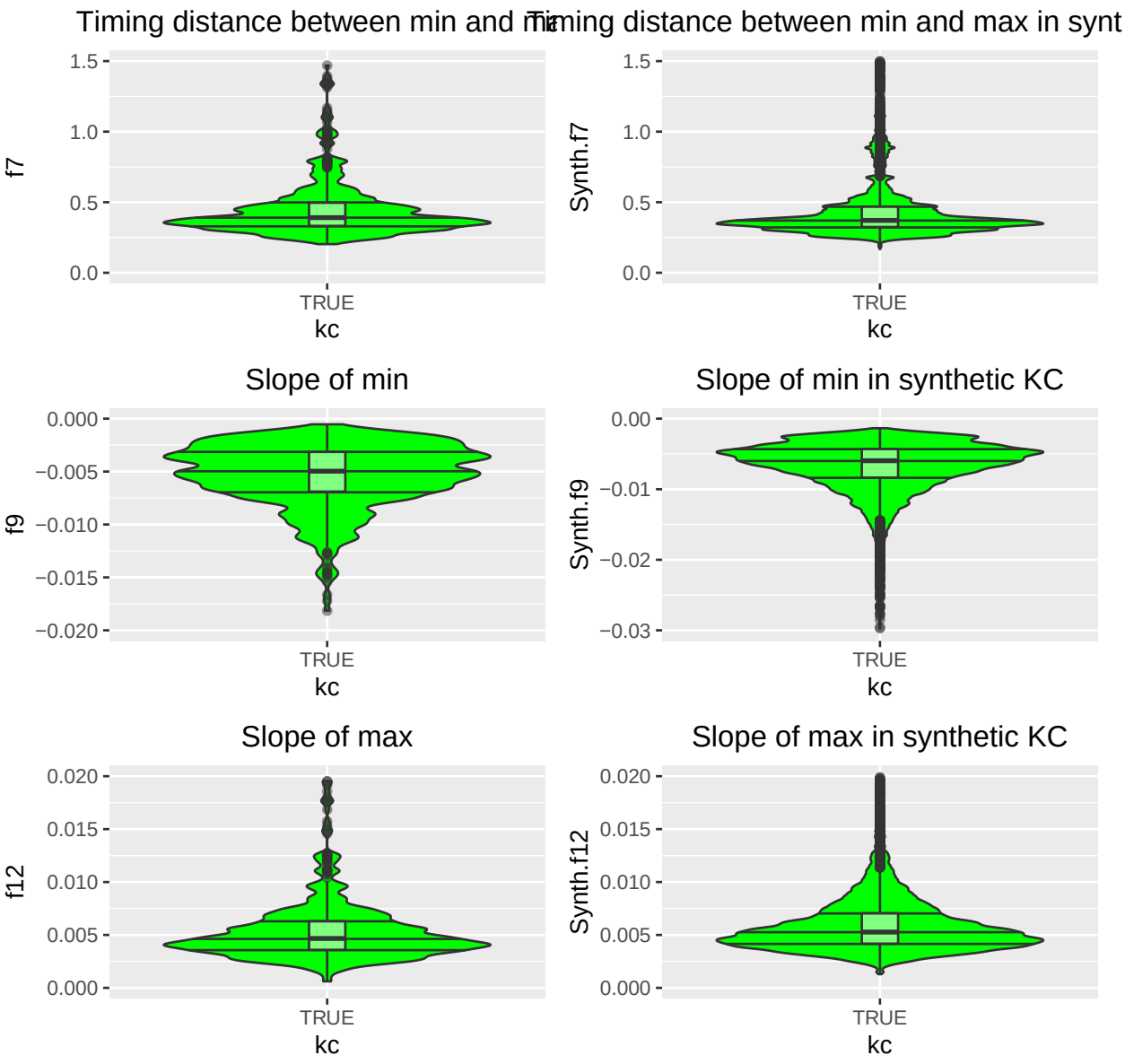
Features by persistence diagrams

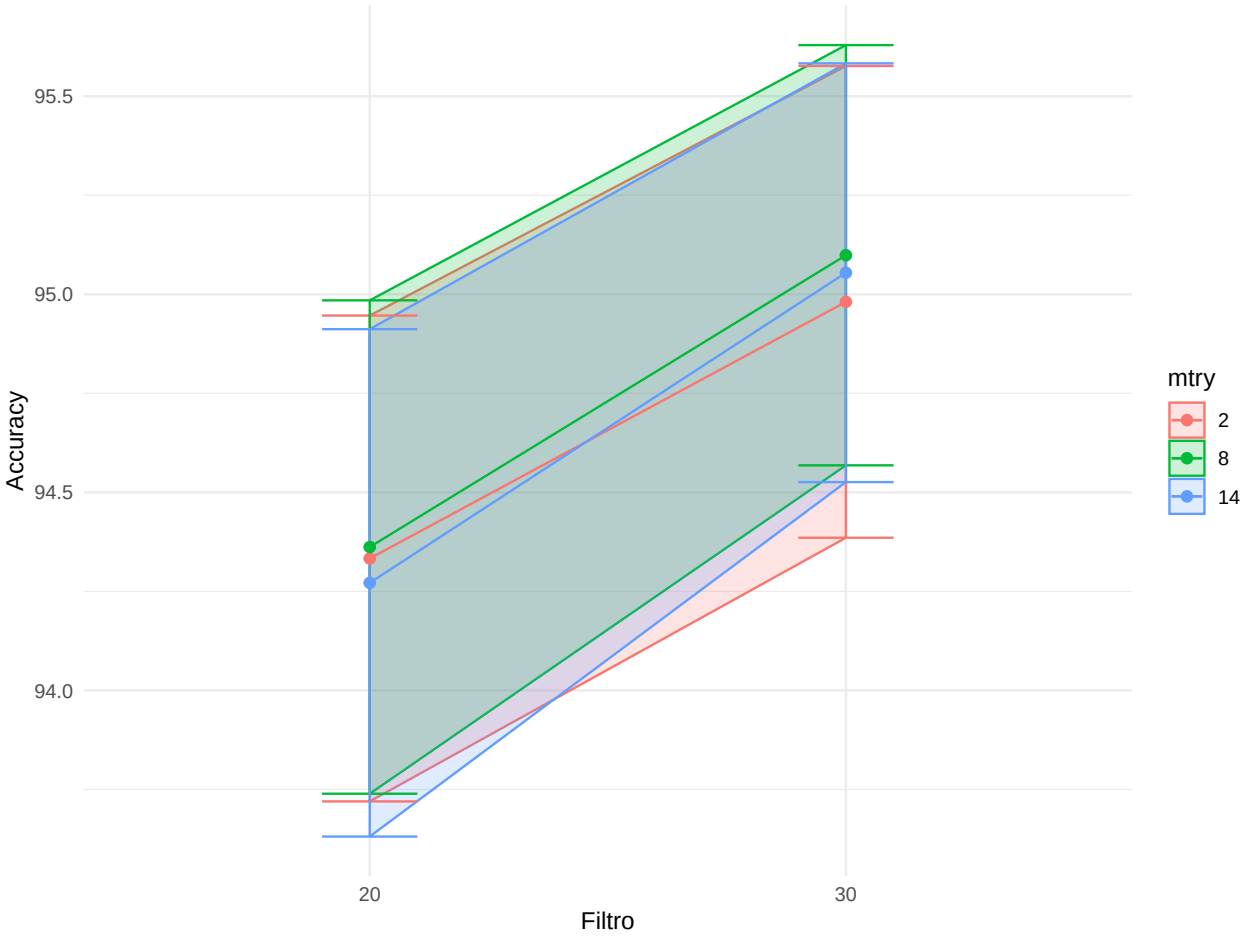


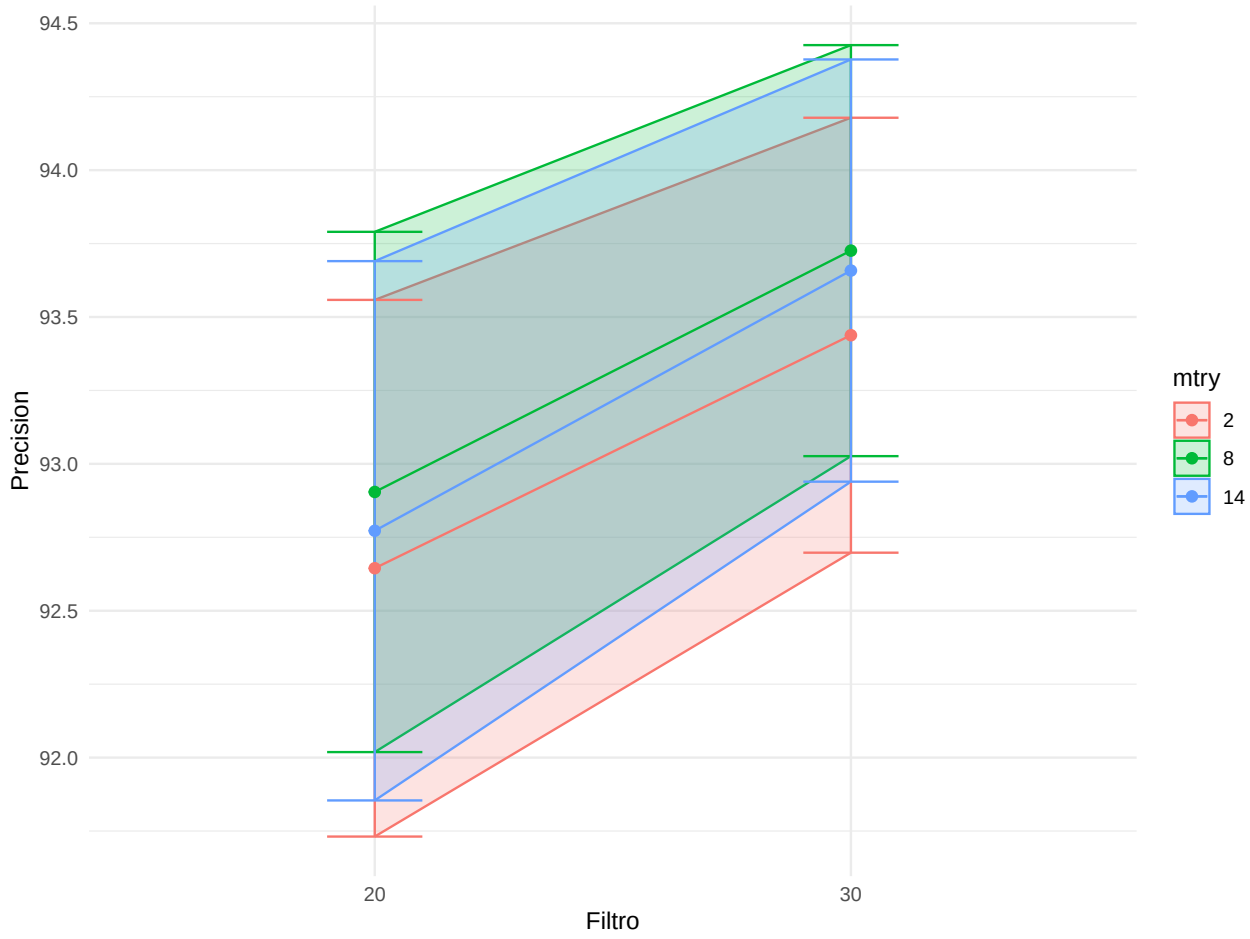
Features by information on the plot

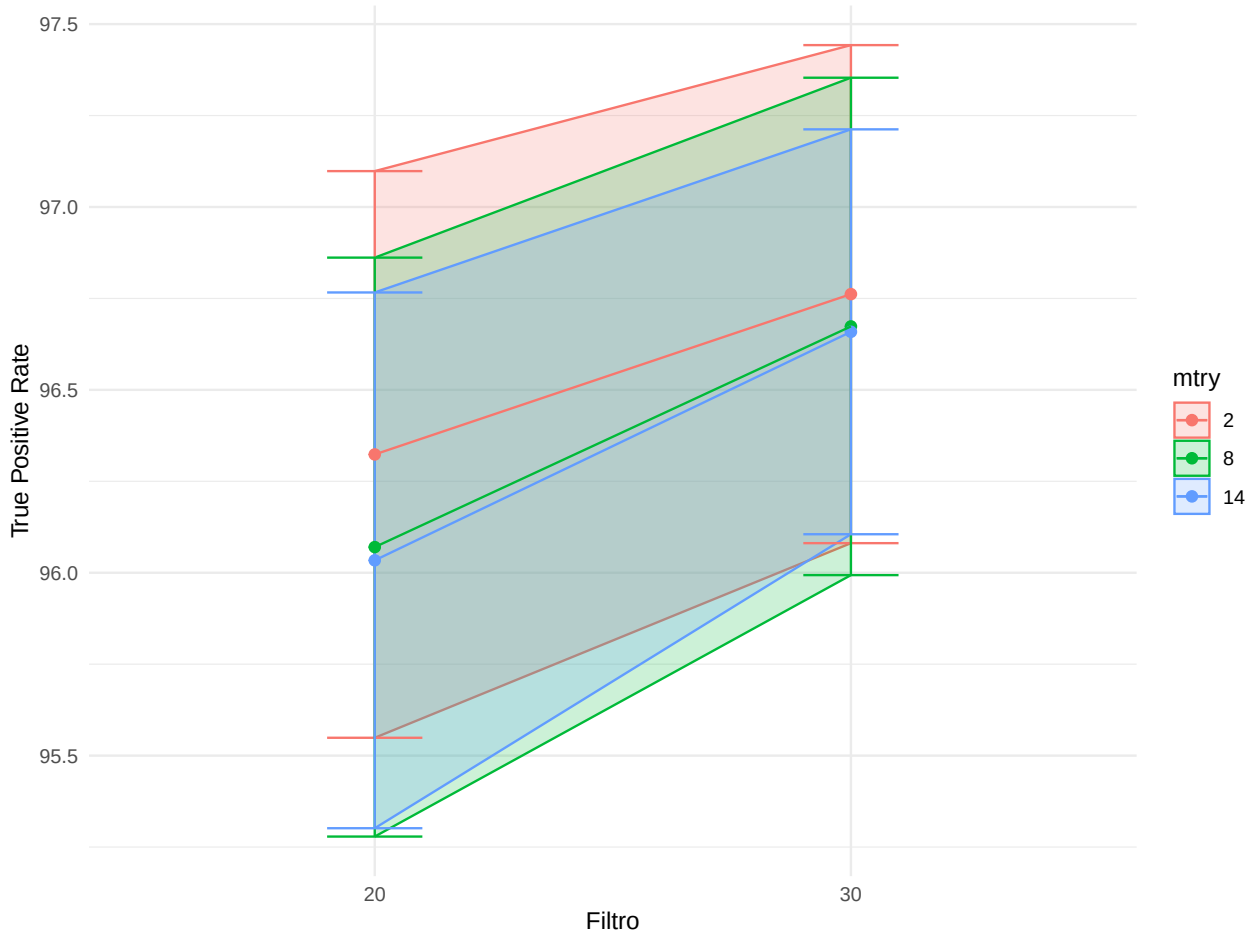


Features by information on the plot




95.5
95.0
94.5
94.0
Accuracy
20
30
Filtro
mtry
2
8
14


94.5
94.0
93.5
93.0
92.5
92.0
Precision
20
30
Filtro
mtry
2
8
14


97.5
97.0
96.5
96.0
95.5
True Positive Rate
20
30
Filtro
mtry
2
8
14

4.1.2 Fattore di rimescolamento

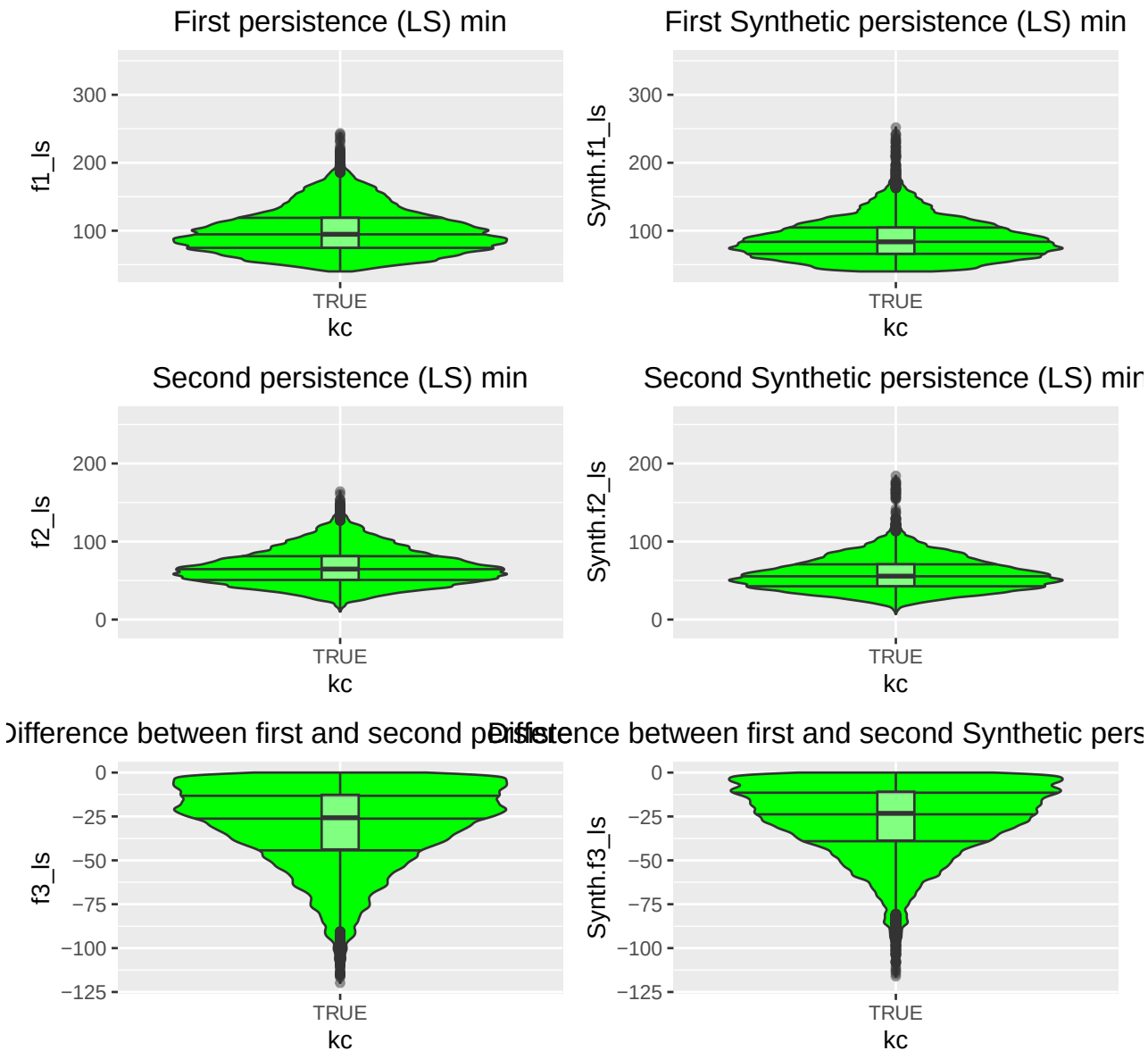
Il fattore di rimescolamento, nel codice identificato come "C" indicizza la percentuale di peso da dare al primo dei due segnali da usare nella combinazione lineare per generare i KC-Sintetici.

```
#step 6

    Synth.KC <- ts((c *
    as.numeric(Synth.signal[[1]]$frame)
    + (1-c) *
as.numeric(Synth.signal[[2]]$frame)),
frequency =
frequency(obj@filtered_signal))
```

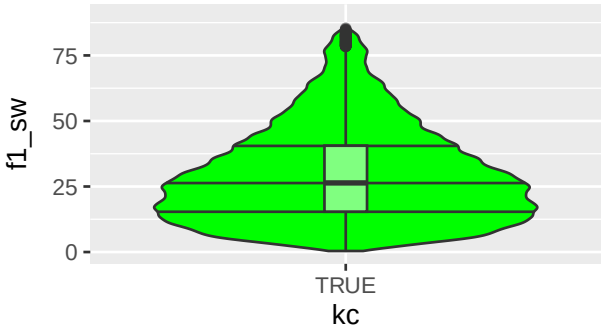
ad esempio nel caso trattato nella sezione precedente il valore scelto per ottenere i sintetici è stato di *"c=0,75"* , questo indica che il primo segnale contribuisce per il 75% al segnale sintetico risultante, mentre il secondo segnale contribuisce per il 25% quindi diamo maggiore peso al primo dei due segnali che inciderà maggiormente sulla combinazione lineare. Nei seguenti Violin Plot si può vedere il confronto tra le features sintetiche (grafici di destra) provenienti da due diversi valori: nelle prime quattro pagine C assume proprio il valore *0,75*. Come detto precedentemente, il primo segnale ha un peso maggiore sulla combinazione lineare. Nelle quattro pagine successive il fattore di rimescolamento è paritario ovvero *0,50*. Si intuisce che la scelta di questo fattore influenza totalmente la qualità del sintetico creato e di conseguenza la sua topologia.

Features by persistence diagrams

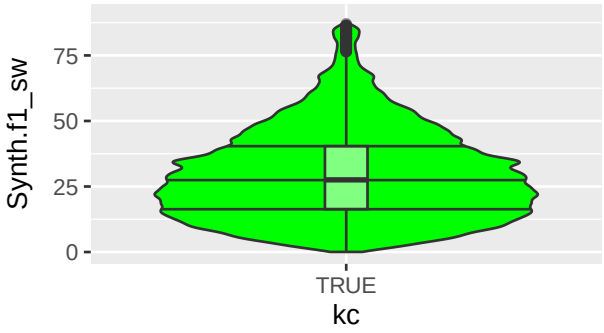


Features by persistence diagrams

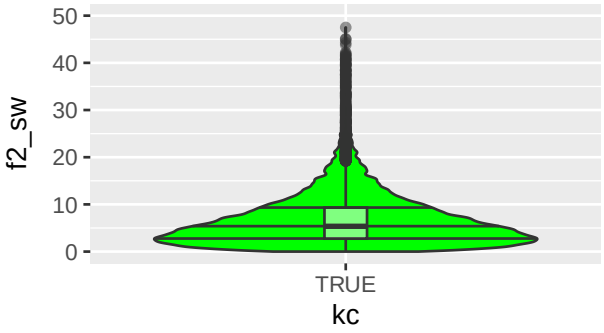
First persistence (SW) min



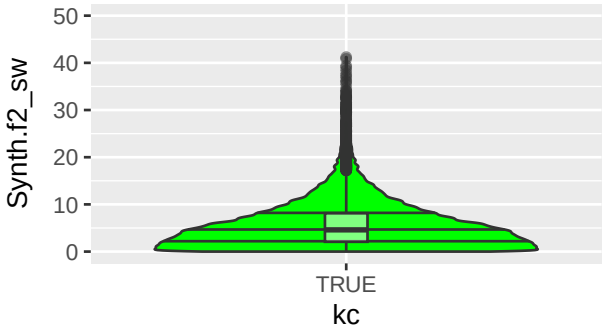
First presistence (SW) min



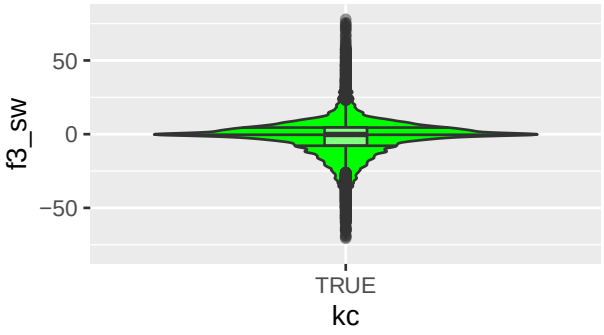
Second persistence (SW) min



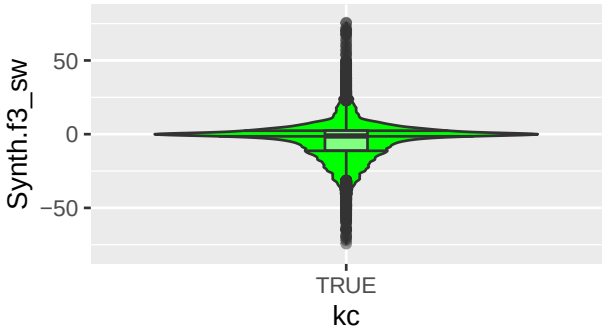
Second persistence (SW) min



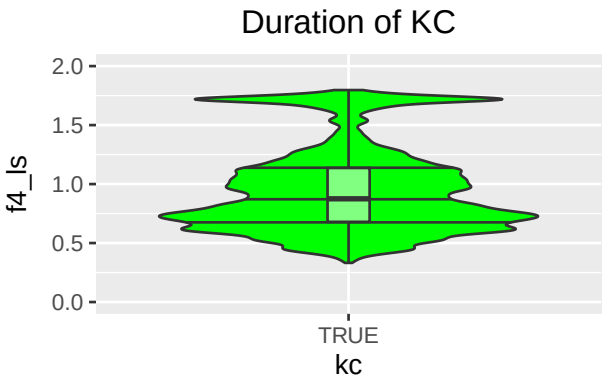
ifference between first and second persist

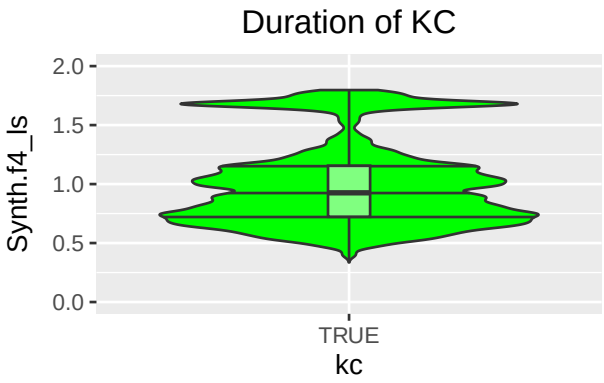


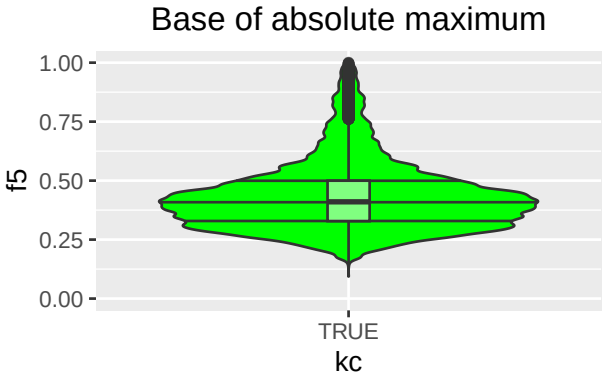
Difference between first and second persistenc

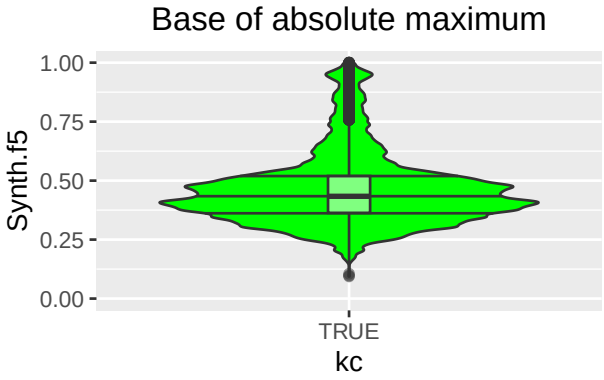


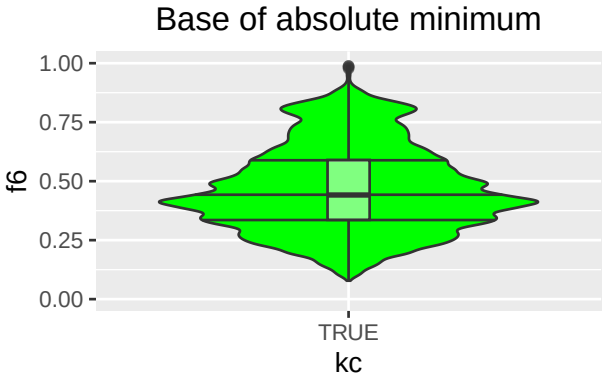
Features by information on the plot

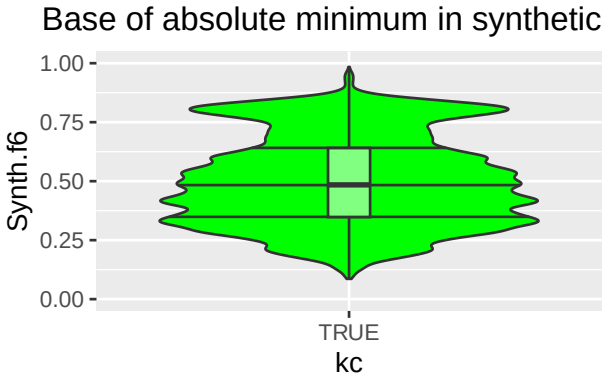




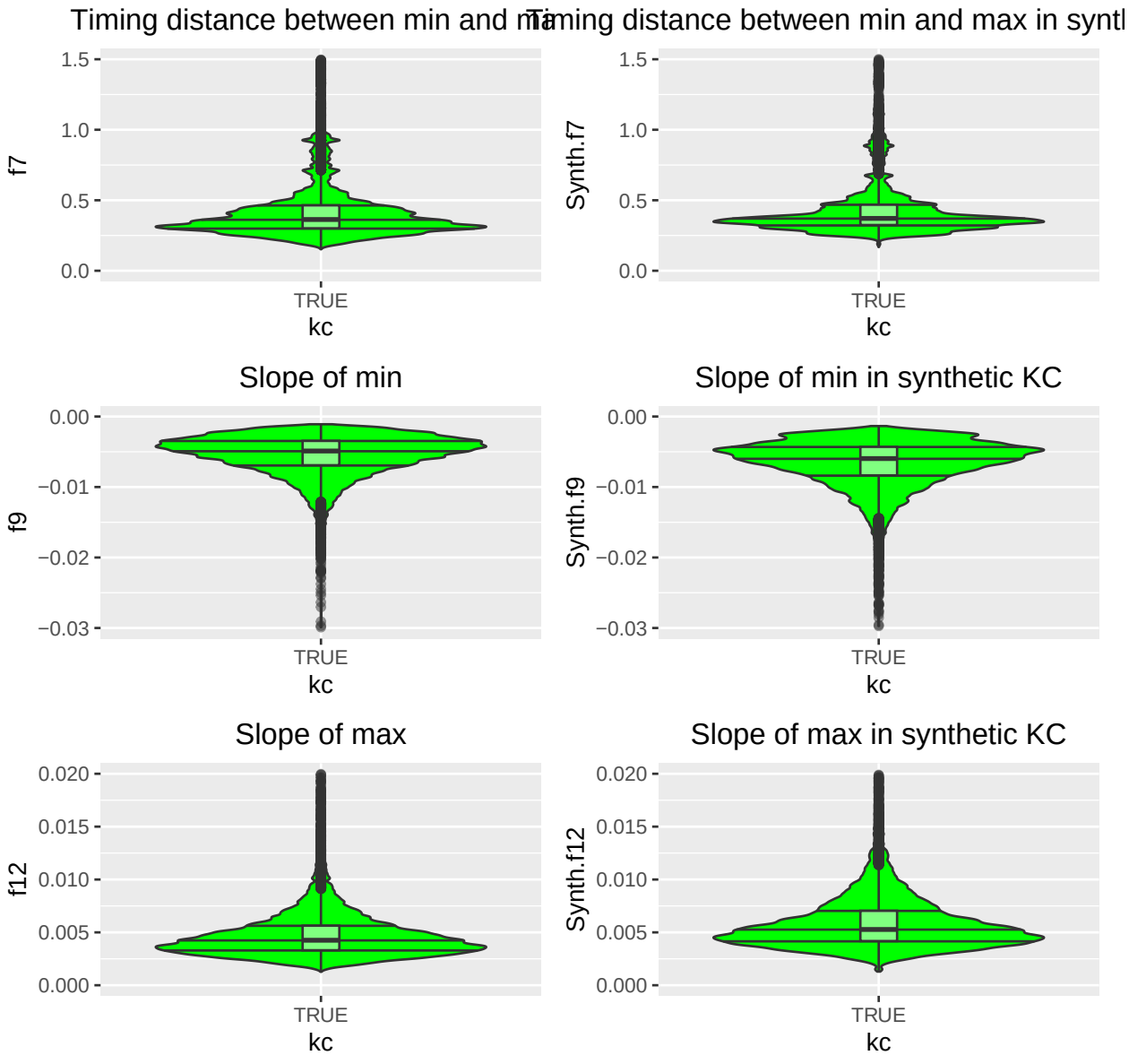




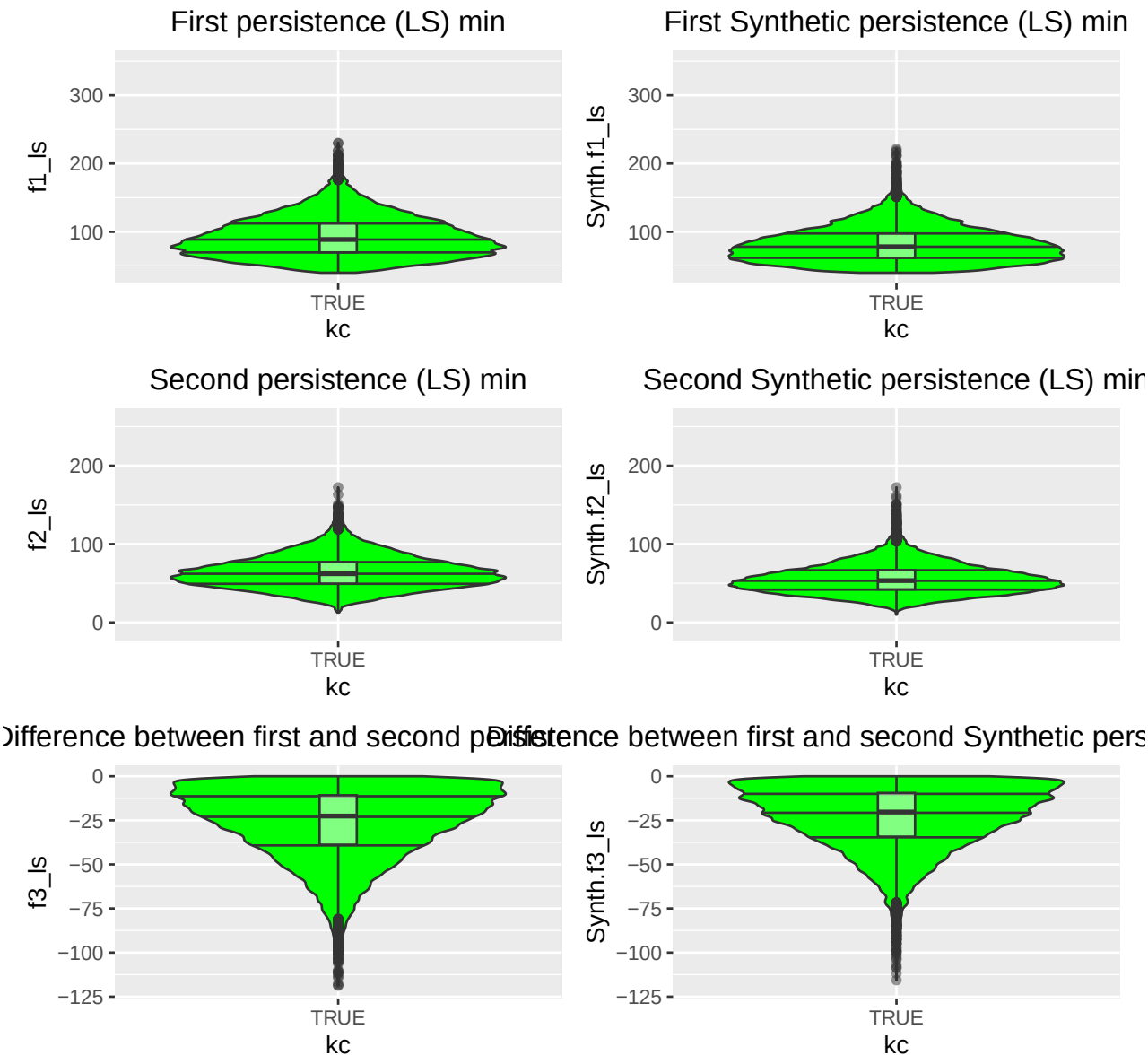




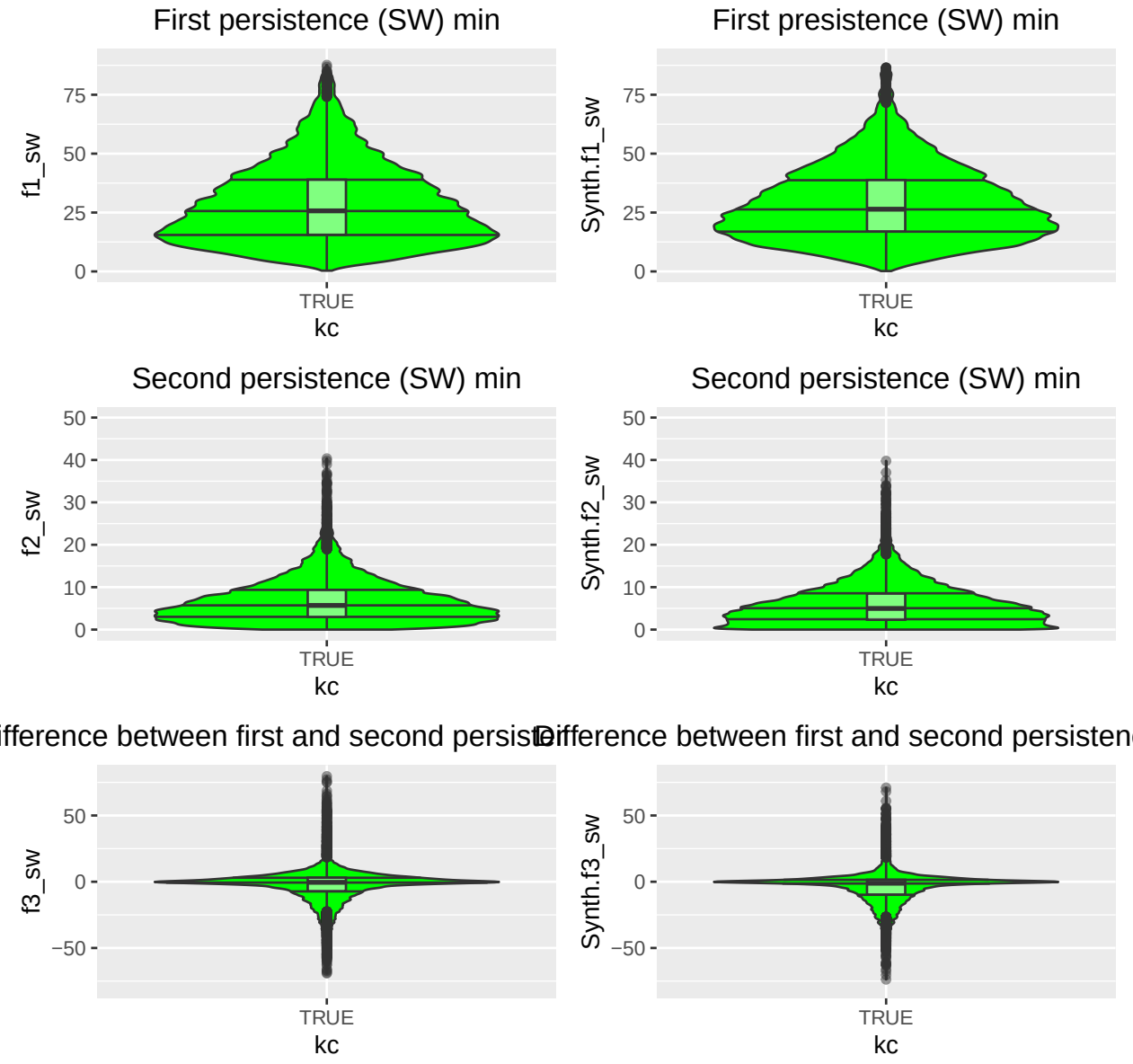
Features by information on the plot



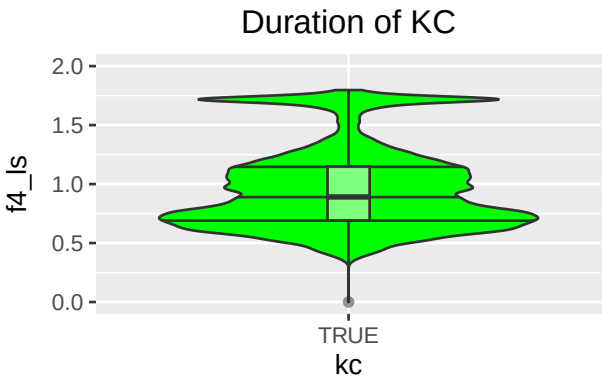
Features by persistence diagrams

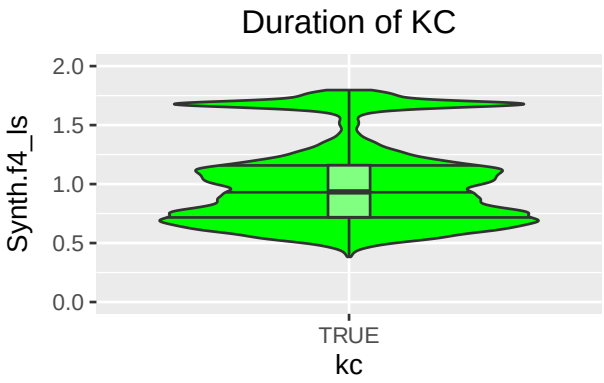


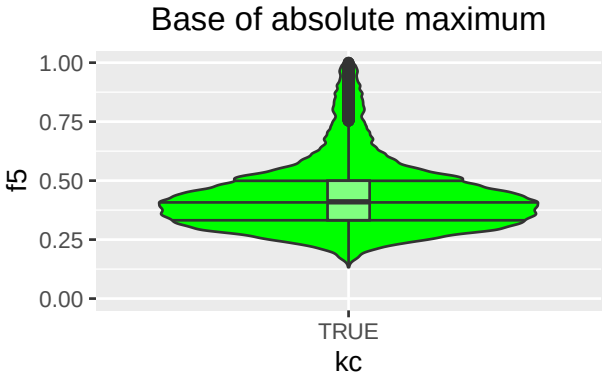
Features by persistence diagrams

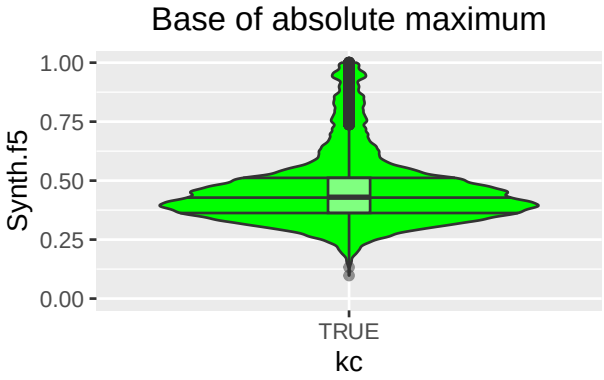


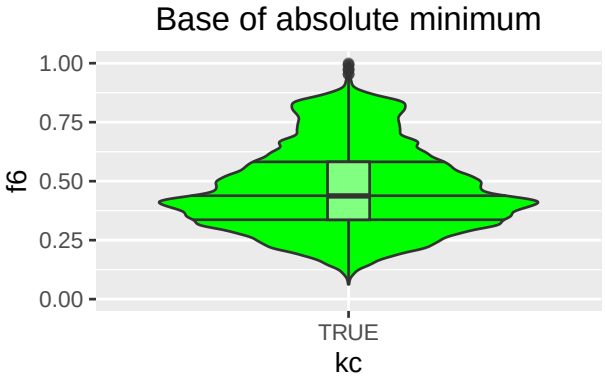
Features by information on the plot

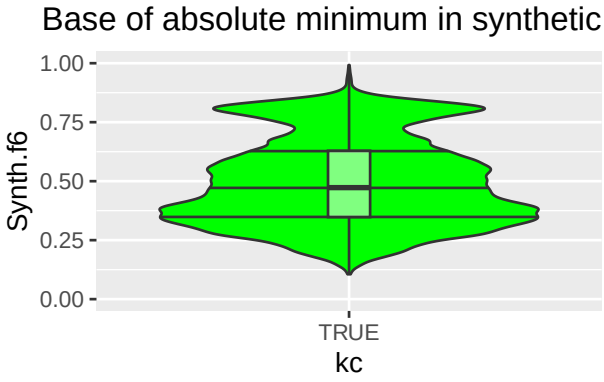




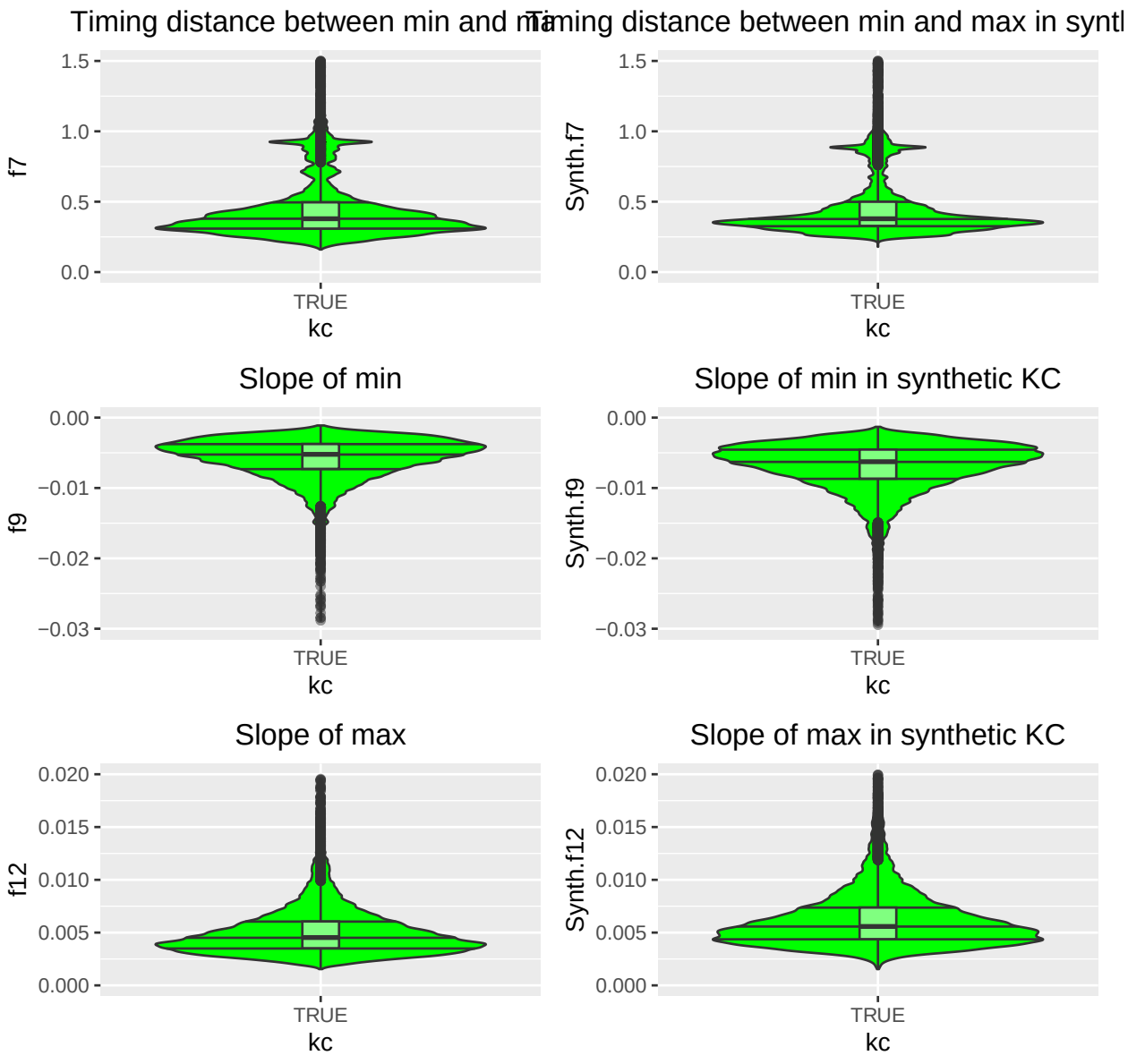








Features by information on the plot



4.2 Peak_distance

All' interno dello Step2 uno dei parametri più importanti è *peak_ distance* . Il parametro *"peak distance"* si riferisce alla distanza minima, da intendere in secondi, richiesta tra due picchi successivi in un segnale. Questo parametro viene utilizzato per identificare i picchi in un segnale (ad esempio un segnale EEG) e permette di controllare quanto vicini tra loro possano essere i picchi trovati. Cambiare questo parametro comporta una variazione nel numero e nella posizione dei picchi identificati. Un valore più basso consente di identificare più picchi, anche quelli che sono vicini tra loro. Questo può essere utile quando si vuole catturare maggiori dettagli nel segnale, ma aumenta il rischio di rilevare falsi picchi, ossia picchi che in realtà non sono significativi.

```
    setMethod("getMin", "SInfo", function(obj,
    peak_distance, min_height, sleep_stage = "Sleep
    stage 2"){
  points <- as.numeric(obj@filtered_signal)

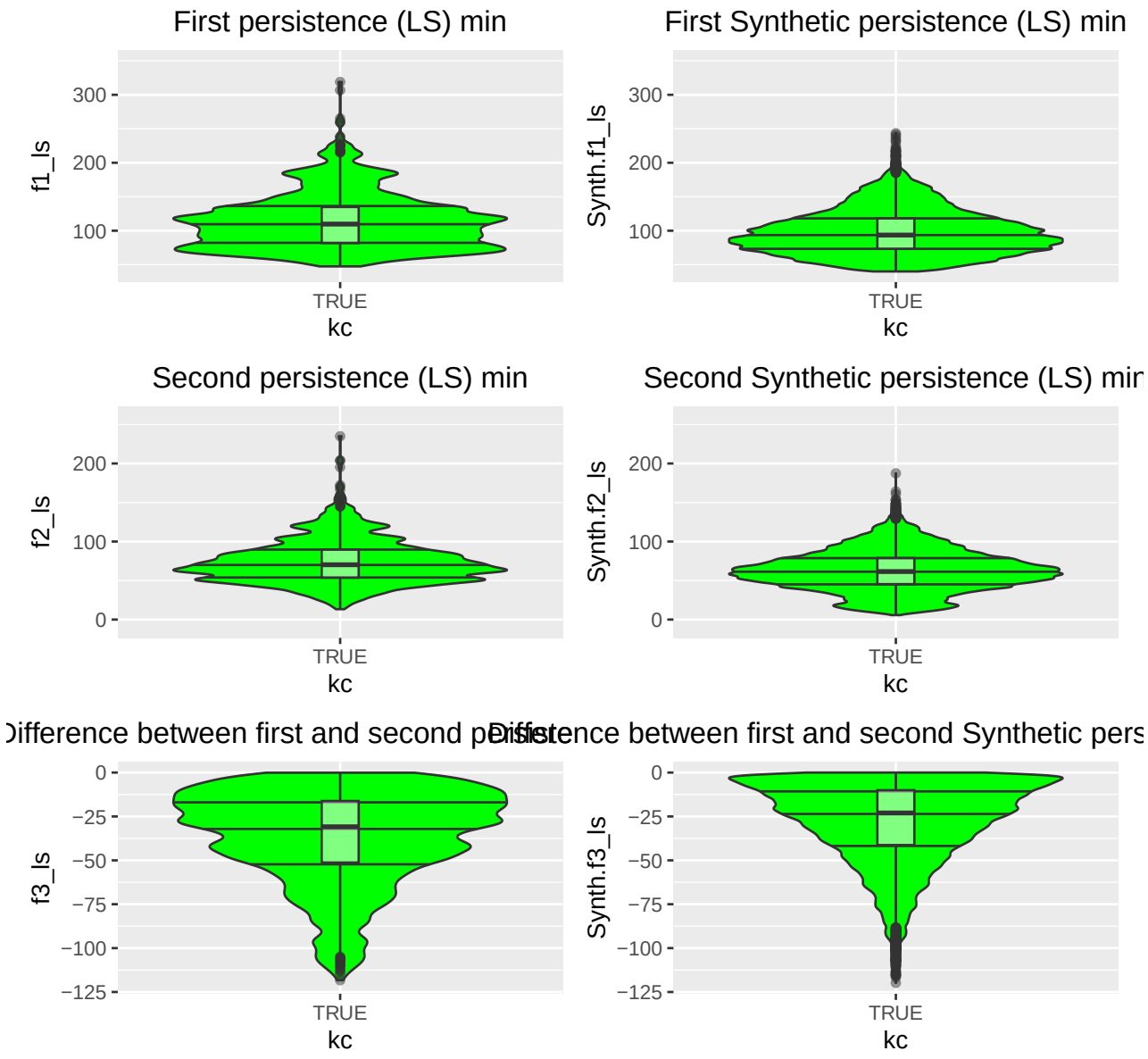
  # converto i secondi che devono intercorrere
  tra un minimo e l'altro in punti
  dist_between_minimum <- peak_distance *
  frequency(obj@filtered_signal)
    [...]
```

Un valore più alto rileva solo i picchi più distanti tra loro, filtrando i picchi ravvicinati. Questo riduce il numero di picchi rilevati e può evitare falsi positivi, ma rischia di trascurare dettagli importanti, specialmente in segnali complessi o rumorosi. Considerando che il valore da passare è al momento richiesto all' utente, e nonostante questo sia presumibilmente un esperto comunque dobbiamo tentare di settare un valore di *peak_ distance* sempre sensato. Il valore di default è *0.50*, i valori testati sono stati: *0,75* e *0,25*

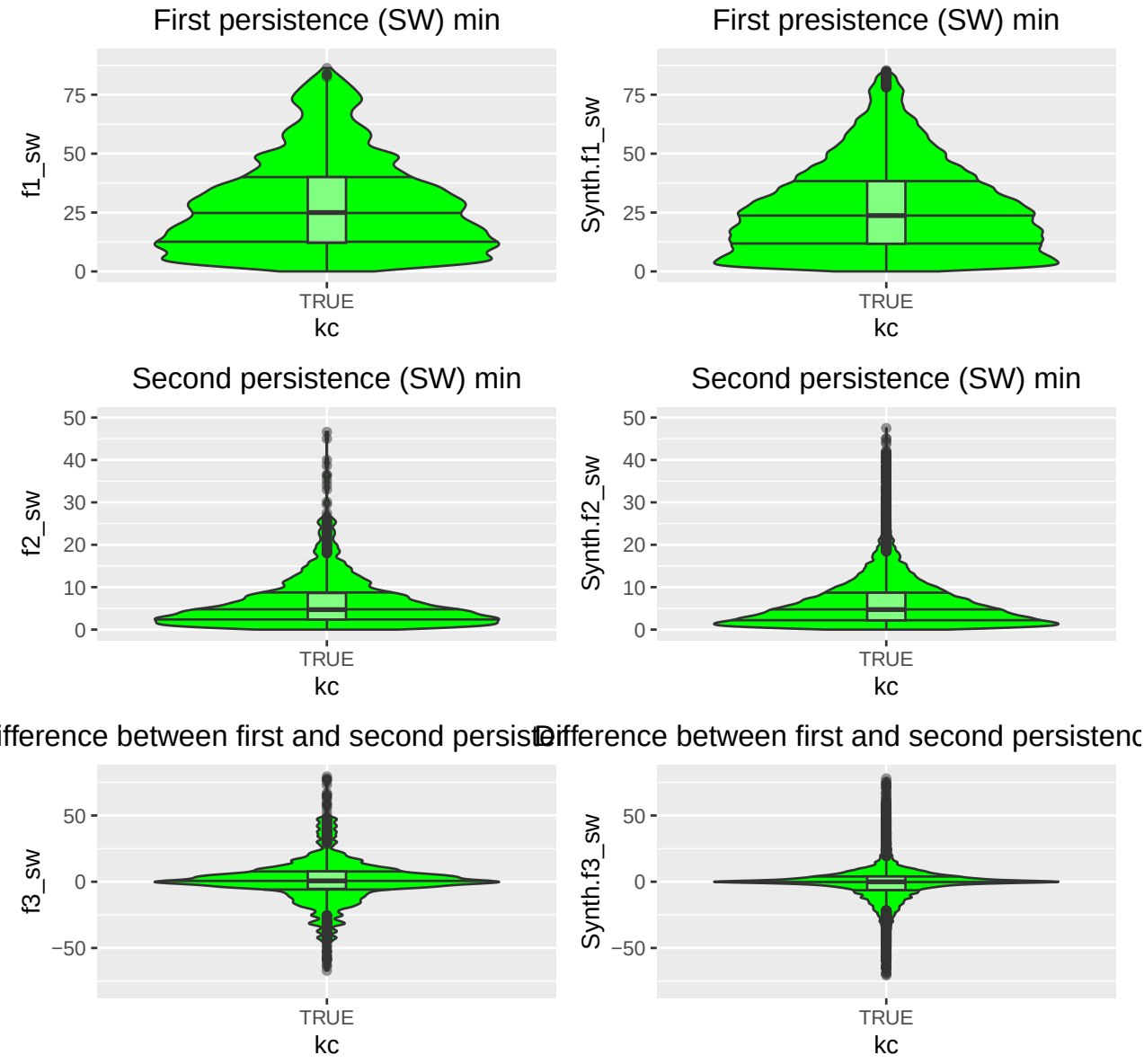
```
# ottengo i minimi
  cat("ottengo minimi", "\n")
  #step2
  KCobj@pot_kc$min_vec <-
  getMin(KCobj, 0.25, 20) /
  getMin(KCobj, 0.75, 20)
```

Regolare il parametro *peak_ distance* permette di trovare un equilibrio tra sensibilità e precisione nella rilevazione dei picchi. Un valore troppo basso può aumentare il rumore nei dati, mentre un valore troppo alto potrebbe far perdere informazioni utili. Nei prossimi plot si possono osservare i plot ottenuti passando il parametro *0,25* e mantenendo un parametro di filtraggio di valore *20*:

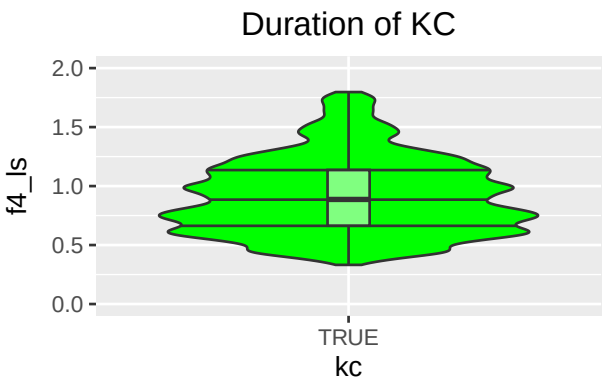
Features by persistence diagrams

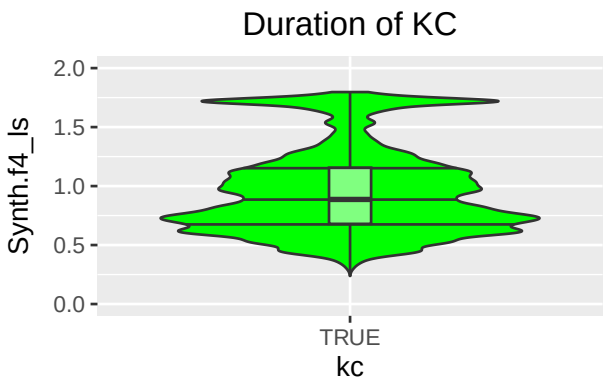


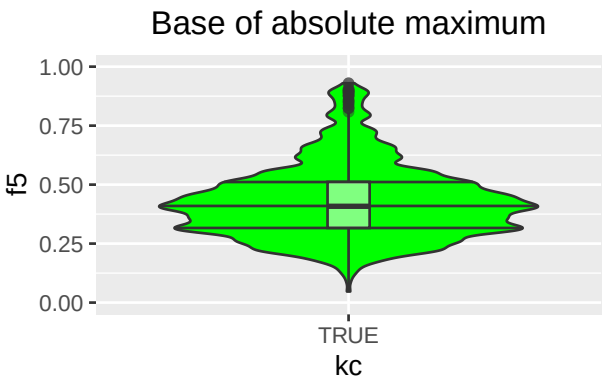
Features by persistence diagrams

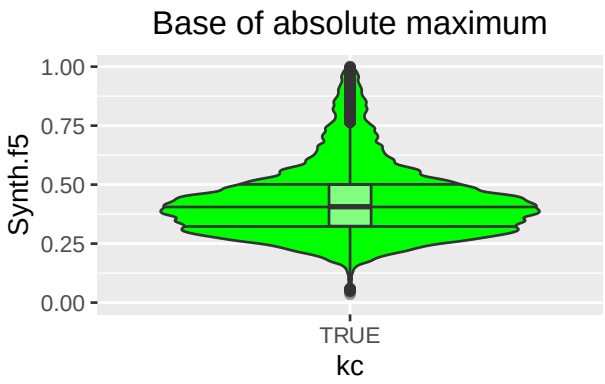


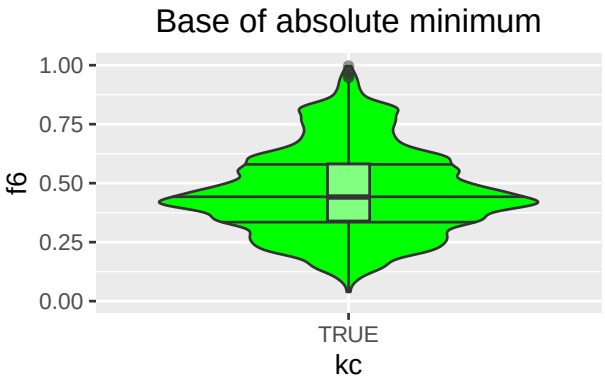
Features by information on the plot

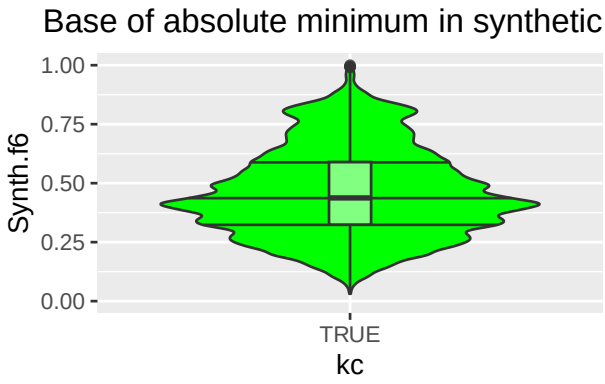




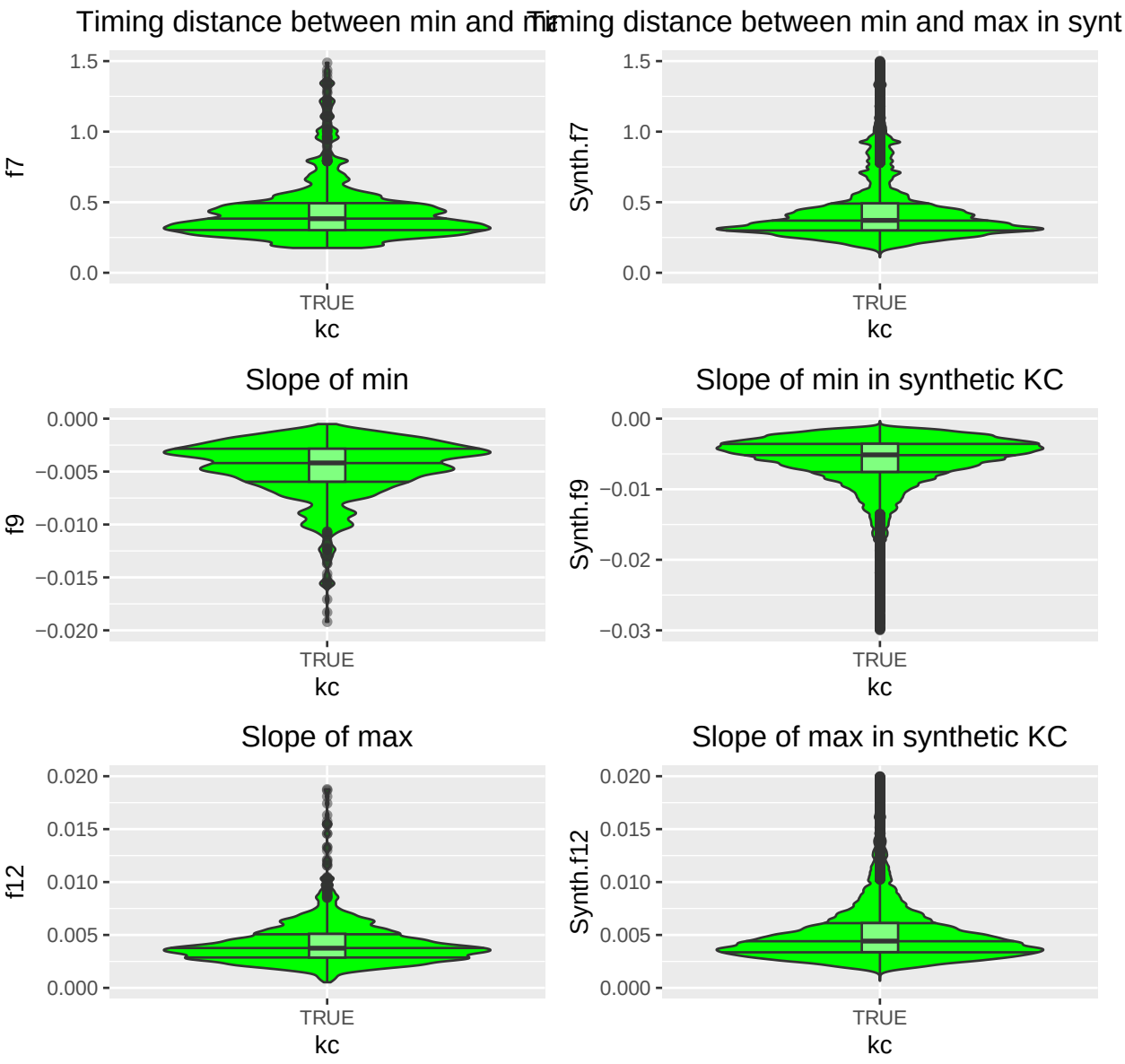








Features by information on the plot



4.3 Parallelizzazione e Garbage Collection

Per concludere il discorso sulle performance, seppur non direttamente correlate alla logica del classificatore, ha senso scendere di livello architettonico, e fare un accenno sulla parallelizzazione e i cluster. La parallelizzazione è un aspetto fondamentale per migliorare l'efficienza computazionale nei progetti che richiedono grandi quantità di elaborazione. In particolare, quando si lavora con la persistenza topologica, il calcolo può risultare intensivo, un esempio nei test è stata su tutte il calcolo della SW che ha richiesto diverse ore di computazione. L'uso della parallelizzazione consente di distribuire il carico di lavoro su più core, riducendo significativamente i tempi di esecuzione complessivi. L'approccio scelto per la parallelizzazione si basa sulla suddivisione del flusso di lavoro in processi indipendenti che possono essere eseguiti contemporaneamente. In questo caso, le diverse configurazioni dei parametri. Nel contesto di R, esistono diverse librerie che facilitano la parallelizzazione. Una delle più utilizzate è *parallel*, che permette di sfruttare i core multipli presenti in un sistema, e *foreach*, che permette di eseguire iterazioni in parallelo su un cluster di core. Il vantaggio di queste librerie è che consentono di scrivere codice che viene parallelizzato senza dover cambiare in modo sostanziale la logica del programma.

```
library(parallel)
cl <- makeCluster(detectCores() - 1) #
utilizzo tutti i core tranne uno
clusterExport(cl, varlist = c("KCobj",
"getMin", "pers_diag",
"extract_features",
"make_forest"))
list.synth <- parLapply(cl, filter_vec,
func)
stopCluster(cl)
```

Questo codice sfrutta tutti i core disponibili tranne uno per eseguire in parallelo la funzione su ciascun valore del vettore *filter_ vec*. In questo modo, il tempo necessario per testare tutte le configurazioni di filtro viene drasticamente ridotto. Questo approccio è stato esteso anche ad altre parti del flusso di lavoro, come l'estrazione delle caratteristiche e il calcolo dei diagrammi di persistenza.

Un altro aspetto importante da considerare è la gestione della memoria. Processi paralleli possono infatti consumare una quantità significativa di memoria, e se questa non viene gestita correttamente, si rischia di esaurirla, causando rallentamenti ed eventualmente il crash del sistema. Per affrontare questo problema dopo computazioni pesanti richiamiamo la "garbage collection" con *gc()*. In R, il metodo gc() crea un processo che libera la memoria non più utilizzata dal programma, rendendola nuovamente disponibile per altre operazioni. Questo è particolarmente rilevante quando si eseguono calcoli intensivi, come quelli richiesti per il calcolo della persistenza o delle fetures, che possono generare grandi quantità di dati temporanei.

```
gc() # chiama il garbage collector
per liberare memoria inutilizzata
```

Integrare queste tecniche di parallelizzazione e gestione della memoria all'interno del flusso di lavoro del classificatore ha consentito di migliorare significativamente

le prestazioni, riducendo il tempo di calcolo e garantendo l'efficienza nell'uso delle risorse hardware. L'ottimizzazione tramite tecniche di parallelizzazione consente di affrontare l'elevata complessità computazionale del progetto senza comprometternee l'accuratezza dei risultati ma anzi migliorando la qualità del calcolo, che senza questi approcci richiederebbe un tempo anche quattro volte superiore.

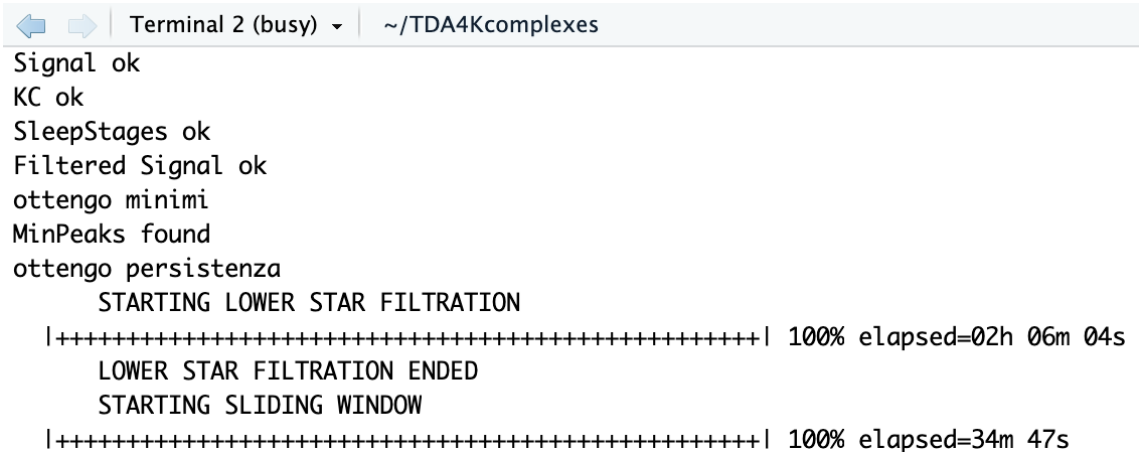


Figure 4.1: tempistiche ottenute durante il calcolo della persistenza tramite parallelizzazione

Chapter 5

Conclusioni

Lo studio condotto ha mostrato l'efficacia dell'approccio usato finora per l'analisi di segnali per lo studio dei KC. Grazie a un attenta revisione e verifica del codice, che è stato prontamente revisionato, e corretto laddove vi è stato bisogno, il classificatore è stato migliorato sotto diversi aspetti. L' ereditarietà delle classi è stata rivista e migliorata, perfezionandone la robustezza e la leggibilità. Il codice in generale si presenta ben strutturato. Verificata l'accuratezza della Random Forest, è stata testata la Cross-Validation come alternativa per creare la foresta ed estrarre le metriche, la CV ha performato bene sui casi testati e si è dimostrata molto affidabile ottimizzando le performance su una suddivisione a 20 fold. Infine è stato iniziato il processo di testing e analisi delle performance, che ha permesso di scartare alcune opzioni parametriche, e confermarne altre. Abbiamo testato il parametro di filtraggio che si occupa di lisciare il segnale, per il momento lo standard era e resta il valore *20*. Abbiamo testato più parametri per il fattore di rimescolamento del segnale per la creazione di sintetici e la *peak_ distance* che incide direttamente sulla classificazione del singolo KC. Tuttavia la valutazione delle performance è un processo lungo, e che come accaduto, può portare alla luce errori indesiderati e non previsti, questo porta alla correzione degli stessi, con conseguenti rallentamenti sulla tabella di marcia, che nel nostro caso significa tempo in più nel rilascio del pacchetto R per l'analisi dei KC, che è appunto il fine ultimo di questo progetto. Gli obiettivi prossimi sono:

- Risoluzione di alcuni problemi legati a SMOTE, utilizzando un nuovo pacchetto.
- Testing di nuovi valori per i parametri della SW in particolare per i parametri: *m, tau, shift*
- Provare diverse dimensioni nella finestra di segnale, che al momento è fissa.
- implementare l'interfaccia grafica tramite il pacchetto *shiny* di R, al fine di rendere "user-friendly" l'utilizzo del classificatore.
- Creazione del pacchetto R, come precedentemente detto.

In conclusione, il classificatore è attualmente *funzionante e funzionale*, ma essendo ancora in fase di testing richiede ulteriori approfondimenti sulle performance. Tuttavia, i risultati ottenuti finora sono promettenti e fanno sperare in una rapida conclusione dei test, e una pronta disponibilità per la comunità scientifica.

Bibliography

[1] Samuele Perrotta, *Un nuovo algoritmo di machine learning per la classificazione di complessi K basato sull'analisi topologica dei dati e implementazioni R*, Università degli Studi di Torino, 2023.

[2] Giorgio Capaccetta, *Il riconoscimento dei Complessi K: TDA dalle features alla classificazione.*, Università degli Studi di Torino, 2024.

[3] Polisonnografia: quando farla, come funziona, preparazione e durata, *https://privato.policlinicogemelli.it/approfondimenti/polisonnografia*.

[4] Giulio Cuniberti, *TOPOLOGICAL DATA ANALYSIS Aspetti teorici e ap- plicazioni al riconoscimento dei complessi K.*, Tesi di Laurea Magistrale in Matematica, 2020/2021.

[5] Polisonnografia, cos'è, come si svolge e dove farla *https://www.auxologico.it/polisonnografia*.

[6] Elettroencefalogramma (EEG) *https://www.humanitas.it/visite-ed-esami/elettroencefalogramma-eeg/*.

[7] Oracle, in cosa consiste il machine learning. *https://www.oracle.com/it/artificial-intelligence/machine-learning/what-is-machine-learning/*.

[8] Cross-validation: evaluating estimator performance *https://scikit-learn.org/stable/modules/cross$_v$alidation.html*.

[9] Cross-validating regression models *https://cran.r-project.org/web/packages/cv/vignettes/cv.html*.

Ringraziamenti

Alla fine della stesura di questo elaborato sono doverosi alcuni ringraziamenti a persone e luoghi, che hanno contribuito direttamente e indirettamente alla realizzazione dello stesso.
È un piacere, prima che un dovere ringraziare la Prof.ssa Sirovich e il Dott. Pernice per avermi seguito nella stesura di questo testo e per aver messo a disposizione le conoscenze e le tecnologie del loro gruppo di ricerca, a supporto di questo stimolante progetto.
Ringrazio caldamente il Dott. Giorgio Capaccetta con cui ho condiviso la ricerca e lo sviluppo di soluzioni per il classificatore. Nel ringraziarlo per il grandissimo supporto, a lui il migliore augurio di proseguire con successo nello studio di questi temi. Ringrazio Torino, che mi ha accolto e che mi ha stimolato e ringrazio la Sicilia, che è il luogo da cui provengo, ed il posto che amo.

Grazie ai miei genitori; i vostri valori mi accompagnano e mi guidano, il vostro carattere mi ispira e mi rende orgoglioso, a voi dedico tutto il mio affetto, tutto il mio amore.

Grazie ai miei nonni; a chi è qui con me per accompagnarmi in un nuovo percorso, e a chi segue i miei passi dalla sua stella, a voi dedico i miei pensieri.

Grazie a tutta la mia famiglia. A quella di sangue che mi segue, che mi supporta, che mi nutre, che mi migliora. Grazie a voi so di non camminare mai da solo.
Mi auguro di poter essere un aiuto per voi, tanto quanto voi siete un faro per me.

Grazie alla mia famiglia acquisita. A chi c'è da sempre, e a chi, arrivato da poco, ha già ritagliato il suo spazio nel mio cuore. A chi è giù in Sicilia, a chi è su a Torino, a tutti i miei amici sparsi per l'Italia, grazie per essere cresciuti con me. Ogni luogo è casa, se c'è anche e solo uno di voi.
Ai fratelli con cui condivido il viaggio. Il cammino è faticoso, ma soddisfacente, accogliete ogni difficoltà e ogni paura, sarà motivo per brindare e gioire di più quando raccoglierete ciò che avete seminato; quel giorno sarò lì a brindare con voi.

A voi tutti un ringraziamento e un augurio.
MORE LIFE.

